

CLM/Modbus Interface

LK12-210

Communication
Link Module

CLM/Modbus Interface

LK12-210
Release independent
03/2013

Copyright, Notices, and Trademarks

Printed in U.S.A. – © Copyright 1996 - 2013 by Honeywell International Sarl.

Revision 1.13 – March 2013

While this information is presented in good faith and believed to be accurate, Honeywell disclaims the implied warranties of merchantability and fitness for a particular purpose and makes no express warranties except as may be stated in its written agreement with and for its customer.

In no event is Honeywell liable to anyone for any indirect, special or consequential damages. The information and specifications in this document are subject to change without notice.

TotalPlant Solution (TPS) System is a U. S. registered trademark of Honeywell Inc. TPS is the evolution of TDC 3000.

Local Control Network, and LCN are trademarks of Honeywell Inc.

TotalPlant is a registered trademark of Honeywell Inc.

Modbus, Modicon, and 984 are trademarks of Modicon Inc.

Microsoft and Windows/Windows 2003 and 2008 are trademarks of Microsoft Corporation.

This document was prepared using Microsoft Word for Windows.

Honeywell Process Solutions
1860 W. Rose Garden Lane
Phoenix, AZ 85027 USA

1-800 822-7673

About This Publication

This document is the interface manual for the TPS Communication Link Module (CLM) and products using the Modbus protocol which have been certified under the Honeywell IAC Multi Vendor Interface Program (MVIP).

The CLM/Modbus interface can be hosted on two different platforms. Both are completely supported by this interface manual.

- **CLM/Modbus** is provided for the original CLM which is hosted on an Application Module (AM) and requires the CLMI or CCP communications boards.
- **APP/CLM Modbus** installs to the Honeywell Application Processing Platform (APP, running Windows 2003 and Windows 2008) using the APP Base Software (AM personality) and actually emulates the CLMI/CCP communications boards used in the AM version of the CLM. The LCN software and user station operating displays have not changed from the AM-based CLM and are installed in the same manner. The board emulation and protocol software is installed in the Windows environment of the APP, and therefore requires additional software and installation and configuration procedures. The basic functionality and operation of the Modbus interface is the same.

Important! With CLM R200.1, CLM is supported on Experion APP (Windows 2008 platform on both 32 bit and 64 bit variants). For more information, see Section 4.2.2 Install to CLM on Experion APP.
--

These versions of the Modbus application package are provided on two media, one for the Windows-based files that are required for the APP/CLM and one for the LCN-based files. Installation procedures determine the version to be implemented. *Note that ordering the CLM/Modbus will deliver the LCN media only, whereas ordering the APP/CLM Modbus will deliver LCN, TPS (Windows) and Experion media.*

The CLM provides a standard communications platform, which allows the TPS and the Experion to interface with a wide variety of devices and networks. With this interface, the TPS station operator, GUS or US, and the Experion station operator have access to points located on a non-TPS and non-Experion network. Access to Modbus devices is transparent to the operator so that the Modbus points appear to be directly connected to the TPS or the Experion.

The complete CLM functionality is segmented into the following four logical layers:

- Process Application
- CLM Platform
- Network Interface
- Communication Protocol

About This Publication, continued

The bottom three logical layers provide the communications foundation for the top layer or Process Application layer. The Process Application layer is the actual user application and is not considered part of the CLM Platform.

This document relates to the bottom two logical layers: Network Interface and Communication Protocol, with a specific orientation to interfacing MVIP certified products to the TPS. This document supplements the *CLM Platform Implementation Manual*, which covers the CLM Platform logical layer, and the portion of the Network Interface logical layer that is not oriented to a specific non-Honeywell network.

This document is intended for the systems engineer who will install and configure the interface between the TPS or the Experion and a Modbus device or network, and the application engineer who will build the database to image the Modbus data.

Support and Other Contacts

United States and Canada

Contact: Honeywell Solution Support Center
Phone: 1-800-822-7673
Calls are answered by dispatcher between 6:00 am and 4:00 pm Mountain Standard Time. Emergency calls outside normal working hours are received by an answering service and returned within one hour.
Fascimile: 1-973-455-5000
Mail: Honeywell TAC, MS L17
1860 W. Rose Garden Lane
Phoenix, AZ, 85027 USA

Europe, Middle East, and Africa (EMEA)

Contact: Honeywell TAC – EMEA
Phone: +32-2-728-2345
Fascimile: +32-2-728-2696
Mail: TAC-BE02
Hermes Plaza
Hermeslaan, 1H
B-1831 Diegem, Belgium
Email: tac-be02@honeywell.com

Pacific

Contact: Honeywell Global TAC – Pacific
Phone: 1300-364-822 (toll free within Australia)
+61-8-9362-9559 (outside Australia)
Fascimile: +61-8-9362-9564
Mail: Honeywell Limited Australia
5 Kitchener Way
Burswood, WA, 6100, Australia
Email: GTAC@honeywell.com

India

Contact: Honeywell Global TAC – India
Phone: +91-20- 6603-2718/19
1800-233-5051
Fascimile: +91-20- 6603-9800
Mail: Honeywell Automation India Ltd
56 and 57, Hadapsar Industrial Estate
Hadapsar, Pune –411 013, India
Email: Global-TAC-India@honeywell.com

Korea

Contact: Honeywell Global TAC – Korea
Phone: +82-80-782-2255 (toll free within Korea)
Fascimile: +82-2-792-9015
Mail: Honeywell Co., Ltd
4F, Sangam IT Tower B4-4 Block
1590, DMC Sangam-dong, Mapo-gu
Seoul, 121-835, Korea
Email: Global-TAC-Korea@honeywell.com

People's Republic of China

Contact: Honeywell Global TAC – China
Phone: +86- 21-2219-6888
800-820-0237
400-820-0386
Mail: Honeywell (China) Co., Ltd
33/F, Tower A, City Center, 100 Zunyi Rd.
Shanghai 200051, People's Republic of China
Email: Global-TAC-China@honeywell.com

Singapore

Contact: Honeywell Global TAC – South East Asia
Phone: +65-6823-2215
Facsimile: +65-6445-3033
Mail: Honeywell Private Limited
Honeywell Building
17, Changi Business Park Central 1
Singapore 486073
Email: GTAC-SEA@honeywell.com

Japan

Contact: Honeywell Global TAC – Japan
Facsimile: +81-3-6730-7228
Mail: Honeywell K.K.
New Pier Takeshiba, South Tower Building
20th Floor, 1-16-1 Kaigan, Minato-ku
Tokyo 105-0022, Japan
Email: Global-TAC-JapanJA25@honeywell.com

Elsewhere

Call your nearest Honeywell office.

World Wide Web

Honeywell Process Solutions website:

<https://www.honeywellprocess.com/>

Training Classes

Honeywell Automation College:

<http://www.automationcollege.com>

Table of Contents

SECTION 1 – INTRODUCTION	13
1.1 Overview.....	13
SECTION 2 – COMMUNICATION LINK MODULE	17
2.1 Communication Link Module Logical Structure	17
SECTION 3 – MODBUS DATA STRUCTURES	19
3.1 Introduction	19
3.2 Building Modbus Interface Structures	19
3.2.1 Building Network (NDT) Structures	20
3.2.2 Building Point Interface (PIT, PITG, PITE) Structures.....	21
3.3 Required Files For Modbus Interface.....	22
3.4 CLM/Modbus Interface Package – Specific Contents.....	23
SECTION 4 – MODBUS INSTALLATION.....	25
4.1 Overview / Setup.....	25
4.2 Install to CLM on APP	26
4.2.1 Install to CLM on TPS APP.....	27
4.2.2 Install to CLM on Experion APP.....	34
4.3 Install to LCN.....	41
4.3.1 Move Files to LCN.....	42
4.3.2 Compile the LCN Pictures	43
4.4 Configure the CLM Loader Directive File.....	44
4.4.1 Configure the APP/CLM (APP) Loader Directive File.....	45
SECTION 5 – MODBUS INTERFACE DESIGN	49
5.1 Overview.....	49
5.2 Network (NDT) Design and Configuration	51
5.2.1 CLMI – Define and Configure the NDT	53
5.2.2 Port – Define and Configure the NDT	54
5.2.3 Device – Define and Configure the NDT	58
5.3 NDT Status Detail Displays.....	61
5.4 PITG and PIT – Define and Configure	63
SECTION 6 – MODBUS IMAGE POINT INTERFACE DESIGN.....	65
6.1 Overview.....	65
6.2 Modbus Data Types and Addresses.....	67
6.2.1 Modbus Data Formats	70
6.3 LCN/Modbus Conversion Codes.....	73
6.4 PITE Level Options	80
6.5 Modbus Image Point Types	81
6.5 Modbus Image Point Types, continued	82
6.5.1 Analog Input Point (AM Regulatory Point Type).....	83
6.5.2 Analog Output Point (AM Regulatory Point Type)	86
6.5.3 Accumulator Point (Rate and Accumulation).....	90
6.5.3.1 Accumulator Point (Register = Rate Value).....	92
6.5.3.2 Totalizer Point (Register = Accumulated Value).....	96
6.5.4 Numeric Point (AM Numeric Point Type)	100
6.5.5 Digital Input (AM Switch Point Type)	102
6.5.6 Digital Composite (AM Switch Point Type).....	104
6.5.7 Flag Point (AM Flag Point Type).....	106

6.5.8	Message Point (AM Custom Point Type)	108
6.5.9	Array Input / Output (AM Custom Point Type), continued.....	112
SECTION 7 – MODBUS IMAGE POINT OPERATION		117
7.1	Overview.....	117
7.2	Operation – Analog Input.....	118
7.3	Operation – Analog Output	119
7.4	Operation – Accumulator (Register = Rate value).....	120
7.5	Operation – Accumulator (Register = Accumulated value).....	123
7.6	Operation – Numeric	124
7.7	Operation – Digital Input.....	125
7.8	Operation – Digital Composite.....	126
7.9	Operation – Flag.....	127
7.10	Operation - Message Point	128
SECTION 8 – REFERENCES		131
8.1	Reference Documents.....	131
APPENDIX A – NDT – NETWORK DEFINITION FILE TEMPLATES		133
A.1	Understanding NDT Template Definition Files	133
APPENDIX B – PITE TEMPLATE DEFINITION FILES		135
B.1	Point Interface Templates.....	135
APPENDIX C – CL INTERFACE TO NON-HONEYWELL DEVICE		139
APPENDIX D – ERROR MESSAGES / FUNCTION CODES		141
D.1	Modbus Function Codes.....	141
D.2	Modbus Error Codes.....	141
D.3	CLM Platform / Modbus Error Codes	141
APPENDIX E – UCLM: CCP AND UCIM REDESIGNED.....		143
E.1	Ordering Information	143
INDEX		145

Acronyms

AM.....	TPS Application Module
APP.....	Application Processing Platform
BIF	Builder Input File
CBO	Check Before Operate
CCP.....	Communication and Control Processor (board)
CDS.....	Custom Data Segment
CL	Control Language
CLM.....	Communication Link Module
CLMI.....	Communication Link Module Interface (board)
DEB.....	Data Entity Builder
EAPP.....	Experion APP
EB	Exception Build
EC	Execute Command (file)
GUS	Global User Station
HM	TPS History Module
LCN.....	TPS Local Control Network
MVIP	Multi-Vendor Interface Program
NCF.....	Network Configuration File
NDT.....	Network Definition Table
PIT	Point Interface Table
PITE	Point Interface Table Entry
PITG.....	Point Interface Table Group
PPS.....	Process Point Special
RBE.....	Report by Exception
RTU.....	Remote Terminal Unit
STSDF	Status Definition File
TDC.....	Total Distributed Control
TDF.....	Template Definition File
TPN.....	TPS Network (LCN)
TPS.....	TotalPlant Solution System
US	TPS Universal Station
ZC	File Extension – CLM Configuration File
ZI.....	File Extension – Builder Input File
ZS	File Extension – Detail Status Display File
ZT.....	File Extension – Template definition File

Section 1 – Introduction

1.1 Overview

Introduction

This manual describes how to interface devices and networks that communicate via the Modbus protocol to the TPS and Experion System using the Communication Link Module (CLM).

There are two versions available for this CLM/Modbus interface. Both are completely supported by this interface manual.

- **CLM/Modbus** is the original CLM, which is hosted on an Application Module (AM) and requires the CLMI or CCP communications boards.
- **APP/CLM Modbus** installs to the Honeywell Application Processing Platform (APP) using the APP Base Software (AM personality) and actually emulates the CLMI/CCP communications boards used in the AM version of the CLM.

Important! With CLM R200.1, CLM is supported on Experion APP (Windows 2008 platform on both 32 bit and 64 bit variants). For more information, see Section 4.2.2 Install to CLM on Experion APP.
--

What this manual does

The two supported versions of the interface have some differences and where necessary, will be clearly identified. This manual documents the CLM network interface layer that defines the Modbus network and the devices connected to it. The manual describes the AM image points used to reflect the data from the Modbus device. The communications protocol that is used to move the data between the Modbus coils and registers and the CLM is documented in the *Modicon Modbus Protocol Reference Guide*. The manual also includes information that supplements the *CLM Platform Implementation Manual* with specific details regarding the Modbus interface network.

What it does not do

This manual does not document how to write specific process applications using the Modbus network data. Refer to the *CLM Platform Implementation Manual* for any additional information that is required to complete a CLM/Modbus Interface implementation. Refer to the appropriate vendor manuals for process application information.

Continued on next page

1.1 Overview, Continued

Intended audience	This manual is used by the Network Interface Design Engineer responsible for the definition of a Modbus interface and its process data for use by the TPS Operator, Experion Operator, and the Process Application Engineer.
Prerequisite knowledge	Since this manual supplements the <i>CLM Platform Implementation Manual</i>, you should first become familiar with the CLM Platform manual, especially the following sections: • Building CLM Interface Structures - Section 5 • Defining Interface Structure Templates - Section 6 • Network Interface Design Considerations - Section 7 The user must also be familiar with the Modbus device network.
Two CLM Platform Software Versions	Each of the two versions of this package requires a different installation procedure for the CLM Platform: For APP installation, <i>TPS/CLM Platform Planning and Installation</i> For AM installation, <i>CLM Platform Implementation Manual</i> .
CLM on an APP	The APP/CLM modifies the AM personality of the APP. The CLM software installed to the Windows environment emulates up to two CCP boards. Four communication ports can be emulated per board. Modbus may be configured on multiple ports.
CLM on an AM node	The CLM is a modified Application Module (AM) that contains a maximum of two CLMI or CCP boards. Each of these board types has four communication ports. Multiple Modbus ports on one board are supported. Both point-to-point and multidrop connections are possible under the Modbus protocol.
AM point defines image point	The CLM/Modbus interface provides the station operator access to information located in a Modbus device. The CLM definition uses AM point types, with Custom Data Segments (CDSs), to define image points that represent the information in the device.
Parameter changes by Operator	The station operator can monitor and change selected operating parameters in the device by changing the associated parameters in the CLM image point.

1.1 Overview, Continued

APP/CLM Network Connections

A Modbus network may be physically connected to the APP/CLM via an RS-232 COM port(s) on the APP or by use of a serial port expander card, such as the Stallion Port Expander.

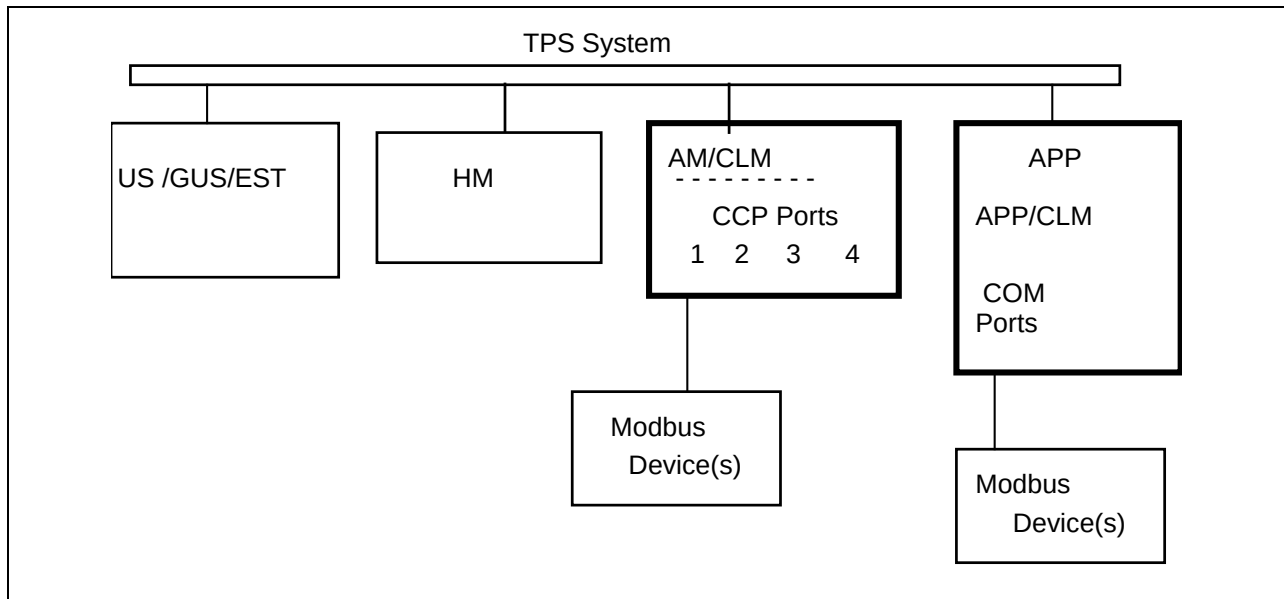
Note: With the R60.12, CLM is supported on Windows 2003. The SYSTECH COM Port Extender needs to be used in place of Stallion card.

CLM/AM Network Connections

A Modbus network is physically connected to the CLM via a paddleboard with an RS-232 or RS-485 link to one or more ports

Figure 1.1-1 presents an overview of an interface between a Modbus network and devices and the TPS System.

Figure 1.1-1 Overview of the CLM Interface to Modbus



Connection to Modbus

The asynchronous connection may be any of the following:

- Direct-connect, using a null modem cable to a single local Modbus device
 - Modems, using short-haul or fiber-optic
 - Multidrop connections to several devices using RS485
 - Multidrop connections utilizing bridges or multiplexers
-

Section 2 – Communication Link Module

2.1 Communication Link Module Logical Structure

Introduction	The CLM interface supports a user application that is monitoring a process and may include control points on a Modbus device.
Four CLM layers	The Communication Link Module consists of four logical layers. The following is a brief definition of those layers.
Process application layer	The process application layer represents the actual application that is monitoring and controlling a process on the Modbus network. It consists of process application points, Control Language programs, and user displays that are built to monitor and control a process. The process application is built on top of the other three layers, and is the end object of the CLM interface.
CLM Platform layer	The CLM Platform layer is the common layer that exists in all CLM environments and consists of software and/or hardware. The platform provides a base for building a gateway to the TPS and Experion System.
Network Interface layer (Network Memory)	The Network Interface layer defines the network and connected devices. It also includes the image points used to reflect the data on the network, and the data conversion information. This Network Interface layer is the subject of this user's manual. Refer to the <i>CLM Platform Implementation Manual</i> for additional information required to complete an implementation.
Communication protocol layer	The communication protocol moves the data between the device control point and the CLM image point for display and use by the upper layers.

Section 3 – Modbus Data Structures

3.1 Introduction

Introduction

Registers and coils are the basic Modbus data structures. This section describes the entries that need to be created to define to the CLM the topography of the Modbus network.

3.2 Building Modbus Interface Structures

Introduction

All CLM interface structures are defined using the CLM Configuration Builder, which is part of the CLM Platform software. Elements include:

- Network Definition Table (NDT)
- Point Interface Table Group (PITG)
- Point Interface Table (PIT)
- Point Interface Table Entry (PITE)

A complete description of the CLM Configuration Builder and the build process is included in the *CLM Platform Implementation Manual*. This section provides a summary of the steps required to complete the definitions of the interface structures.

Builder

The CLM Configuration Builder (referred to as the Builder) will accept input definition from a file or on-line from the station keyboard. The input file mode requires a Builder Input File (BIF) for each interface structure being defined.

BIF files on the HM

By default, the builder looks for BIF files in the NET>&CUS directory. Alternately, BIF files may be placed in a user-named directory. In this case, you must include the directory name in front of the file name when you enter it in the builder (for example, USER>BIF.ZI).

BIF files created before actual build process

BIF files may be created in advance using the TPS System file editor or another text file editor. The BIF files are then transferred to the HM directory stated above for use by the Builder during the actual build process. Refer to Sections 5 and 6 of this document for additional information on the BIF files.

BIF validation with TDF

During the actual interface structure build process, the Builder validates each BIF input file using a pre-defined Template Definition File (TDF) for the specific interface structure type. These TDFs must be available in the HM &CUS directory for exclusive use by the Builder.

3.2.1 Building Network (NDT) Structures

Defining NDT structures

Following is an overview of the steps that describe the creation of the BIF files required for the interface. These steps are used to define and generate the Network Definition Table (NDT) interface structures.

Note: Although this section describes the creation of the input files for the Builder process, entries may be entered on-line one entry at a time as you proceed.

Overview: Steps to Define and Generate NDT Interface Structures

Step	Activity								
1	Identify each addressable component in the network. The number of addressable network components will vary from site to site; however, all networks will have the following types. <table><tr><th>Component</th><th>Description</th></tr><tr><td>CLMI</td><td>CLMI/ CCP Board, 1 or 2 per CLM</td></tr><tr><td>PORT</td><td>1 for each Modbus master connection</td></tr><tr><td>DEVICE</td><td>1 for each Modbus device</td></tr></table>	Component	Description	CLMI	CLMI/ CCP Board, 1 or 2 per CLM	PORT	1 for each Modbus master connection	DEVICE	1 for each Modbus device
Component	Description								
CLMI	CLMI/ CCP Board, 1 or 2 per CLM								
PORT	1 for each Modbus master connection								
DEVICE	1 for each Modbus device								
2	Create a BIF to include one NDT entry for each of the identified addressable network components. • The BIF may be created using the TPS file editor, or other text file editor, and transferred to the History Module (HM) for use. • Names of the BIFs included are listed in Table 3.3-1. • Sample BIFs for the CLMI, Port, and Device are shown in Section 5.2. • Copy the appropriate BIF file and complete each definition within the BIF to create the BIF NDT entry for each addressable network component in your network.								
3	Build the NDT on the target CLM node. This is performed using the CLM Configuration Builder, which is installed on the CLM node as part of the CLM platform software. Instructions for using the CLM Configuration Builder are detailed in the <i>CLM Platform Implementation Manual</i>.								

3.2.2 Building Point Interface (PIT, PITG, PITE) Structures

Introduction

The Point Interface Table Group (PITG) and Point Interface Table (PIT) interface structures are common across all CLM implementations. Therefore, instructions for defining BIF files for these interface structures, and prototype BIF listings, are included in the *CLM Platform Implementation Manual*. This manual will focus on the PIT entries (PITEs) that will be created to image the data from the Modbus network.

Defining PITE structures

Following is an overview of the steps that describe the creation of the BIF files required to build the Point Interface Table Entry (PITE) interface structures.

Note: Although this section describes the creation of the input files for the Builder process, entries may be entered on-line one entry at a time as you proceed.

Overview: Steps to create Point Interface Table Entry (PITE)

Step	Action
1	Identify the specific Modbus points in the network that require an AM image point.
2	Create a BIF for each of the identified image points. • The BIF may be created using the TPS file editor, or other text file editor, and transferred to the HM directory of your choice (default is NET>&CUS) for use. • Names of the prototype BIFs included are listed in Table 3.3-1. • Actual BIF listings for each point type are included in Section 6 of this manual (file extension .ZI). • Copy the appropriate BIF file, and complete each definition within the BIF, to create the BIF PITE entry. • Each PITE entry must be input to the CLM Configuration Builder to generate the PITE interface structure required for each image point in your network.
3	Build the image points and their point interface structures on the target CLM node. This is performed using the TPS Data Entity Builder and the CLM Configuration Builder, which is installed on the CLM node as part of the CLM platform software. • Instructions for using the TPS Data Entity Builder are detailed in the <i>Data Entity Builder Manual</i>. • Instructions for using the CLM Configuration Builder are detailed in the <i>CLM Platform Implementation Manual</i>.

3.3 Required Files For Modbus Interface

Table 3.3-1 Required Files For Modbus Interface

HM NET	Directory	File Type
	&CUS	.Z* , .S
	UCLM	EC, .CL, .AO, .DS

CLM Interface File Types and Extensions

Extension	File Type	Extension	File Type
.AO	Control Language Object Modules	.ZC	CLMI Configuration Modification Sample
.CL	Control Language and CDS	.ZI	Builder Input File (BIF) Samples
.DS	Custom Display Schematics (Source)	.ZS	Status Display Files (CLMI/Port/device/etc.)
.EC	Executable –Install,Link/Unlink CL	.ZT	Builder Template Definition Files (TDFs)
.S	CLMI Load Module		> Network Definition Table (NDT) Templates
			> Point Interface Table Entry (PITE)

Modbus Files supplied on cartridge with interface:				
* - File types EB, DS, and AO may be TPS Release dependent.				
These files will be supplied for each supported release if required.				
Install Modbus to HM			CLMI/CCP	Load Module
	EC		S	
	M_INST		MODB	
Network Configuration Files				
Device	ZT	ZI	ZS	ZC
CLMI				M_CLMI
Port	M_PORT	M_PORT	M_PORT	
MB Device	M_DEVICE	M_DEVICE	M_DEVICE	
Message Point Display			DS*	
			CLMMODB0	
Point Image Files				
ZT	ZI	CL/CDS*	CL/AO	EC
M_AI	M_AI	M_AI		
M_AO	M_AO	M_AO	M_AOCL	M_AO
M_AO2	M_AO2	M_AO		
M_ARYI	M_ARYI			
M_ARYO	M_ARYO			
M_D3	M_D3			
M_D4	M_D4			
M_D5	M_D5			
M_DIN3	M_DIN3			
M_DIN4	M_DIN4			
M_FLAG	M_FLAG			
M_MSG	M_MSG	M_MSG	M_MSGCL	M_MSG
M_NUMER	M_NUMER			
M_TZ	M_TZ	M_TZ		
M_ACCUM	M_ACCUM	M_TZ	M_TZ2CL	M_TZ2

Required CLM Platform Files

ZT	ZI	ZS
CLMI	CLMI	CLMI
PITG	PITG	
PIT	PIT	

Required CLM Platform Files by Point

CL/CDS	CL/CDS	CL/AO	EC
	CLM_CMD3		
	CLM_CMD3		
	CLM_CMD3		
DIG3CDS/CDS1	CLM_CMD3	DIG3CL	DIG3
DIG4CDS/CDS1	CLM_CMD3	DIG4CL	DIG4
DIG5CDS/CDS1	CLM_CMD3	DIG5CL	DIG5
DIN3CDS	CLM_CMD3	DIN3CL	DIN3
DIN4CDS	CLM_CMD3	DIN4CL	DIN4
	CLM_CMD3		
	CLM_CMD3		
	CLM_CMD3		

Point Type	Desc.	AM Point	Point Types	Desc.	AM Point
M_AI	Analog Input	Regulatory	M_DIN3	Digital Input - 3 state	Switch
M_AO	Analog Output (CL Driven)	Regulatory	M_DIN4	Digital Input - 4 state	Switch
M_AO2	Analog Output (Direct)	Regulatory	M_ACCUM	Accumulator	Regulatory
M_ARYI/O	Array Input/Output		M_FLAG	Digital Input / Output	Flag
M_D3	Digital Comp - 3 state	Switch	M_MSG	Message	Custom
M_D4	Digital Comp - 4 state	Switch	M_NUMER	Register Input/Output	Numeric
M_D5	Digital Comp - 5 state	Switch	M_TZ	Totalizer	Regulatory

Note: Each of these EC procedures does an UNLINK of each CL block followed by a LINK of the CL package.

The first time the EC is used, an unlink error will be posted for each CL block.

This initial error should be ignored if the LINK is successful.

end

3.4 CLM/Modbus Interface Package – Specific Contents

Modbus interface package items

The CLM/Modbus Interface Package along with the CLM Platform includes the following items on two release media.

Some modules are specific to either the AM or APP installation.

Documents

- *TPS/CLM Planning, Installation, LK02-200*, for APP installations.
- *CLM Planning, Installation and Service, LK02-100*, for AM installations: Appendix B is specifically for Modbus.
- *CLM Platform Implementation Manual, LK12-340*
- *CLM/Modbus Interface Manual, LK12-210* (this document)
- *CLM Process Operations Manual, LK11-340*

Object Modules

- Platform Software –Object Modules (CLM.LO, CLMI.S, CCP.S)
- Communication Protocol Software – One Object Module (MODB.S)
- APP/CLM adds to or replaces some of these modules.
- Control Language (CL) Object Programs to support device or network specific requirements

Control Language Files

- Custom Data Segment (CDS) Source Packages for custom AM/CLM image points
- Custom Display Package for device or network specific functions
- Unlink/Link CL Blocks to an image point

Template Definition

- Platform Template files for the CLMI NDT layer
- Template files for the Modbus supported NDT layer
- Template files for PIT and PITG (platform templates).
- PITE template file for each image point type
- Prototype BIFs for each template type

Installation Files

- CLMI Configuration File for Modbus Interface
 - Modbus Installation command files to:
 - Copy files to appropriate HM directories
 - Compile all CDS packages included by Honeywell
 - Install APP/CLM to Windows side of APP
-

Section 4 – Modbus Installation

4.1 Overview / Setup

Introduction

The installation of the CLM Platform to either the AM or APP must be completed before continuing. This installation also has different requirements for the AM and APP. After installation you will need to create the AM image points to reflect your network configuration and establish the connection between external network points and the AM image points. (Refer to *CLM Platform Implementation Manual*, Sections 5 and 6.)

The media provided contains files for CLM/Modbus installation to either the AM or APP. Installation of AM related files is the same either to an AM or APP with AM personality. APP installation requires additional procedures for Windows installation.

Prior to installation

1	Read the TPS/TPN Customer Release Guide for information that applies to your installation including HM sizing requirements, special instructions, and reinstallation/migration information.
2	Review the files in the product release, which will be copied to your system to assure no conflicts (duplicate names) in file naming with files already on your system. This is especially critical for CL/CDS files, which may be overwritten. Ref. Table 3.3-1.

APP or AM based CLM?

STOP!

If this is an AM-based CLM, then skip immediately to Section 4.3.

If this in an APP based CLM then continue below with Section 4.2.1 and 4.2.2 first, and then Section 4.3.

4.2 Install to CLM on APP

Overview

There are two parts to the Modbus installation procedure to the APP/CLM.

The first part installs the Windows software on the APP.

The second part installs the AM-based software to the LCN side of the node. The AM-based CLM uses only the second part of the installation procedure, Section 4.3.

After completing this section, continue on with Section 4.3.

Note: For more information on installing CLM modbus on the APP node CLM platform, refer to *TPS/CLM Platform Planning and Installation guide*.

**Stallion or SYSTECH
COM Port Extender
may be required for
multiple ports!**

Important! Connecting a device/network to *two or more* ports may require the use of the Stallion or SYSTECH COM Port Extender interface due to possible lack of available COM ports on the APP/CLM.

4.2.1 Install to CLM on TPS APP

Important! This section is only applicable for users who will continue to use the TPS node on Windows 2003 operating system.

Shutdown TDC Emulators Service

In the Windows environment, the Honeywell TDC Emulators Service simulates the peripheral boards that exist in a traditional TPN node chassis. In the APP/CLM, this service simulates the CLMI and CCP boards found in a traditional CLM. Before installation of a APP/CLM Application, this service should be stopped.

Step	Action
1	On the TPS APP (Windows side of the APP/CLM), log onto Windows as Administrator.
2	From the Start menu select Settings/Control Panel/Administrative Tools.
3	Select Services and in the Services window, scroll down until the TDC Emulators Service appears. See Figure 4.2-1 below.
4	If the Status for TDC Emulators Service is Started, select the TDC Emulators Service entry. From the drop-down menu, select Action, then the Stop button.
5	A message box appears indicating that the service is being stopped. This will take about 30 seconds. Note: Occasionally, the TDC Emulators Service does not stop in the required time and an Error Message box appears with the message “<i>Could not stop TDC Emulators Service service</i>”. When this happens, press the OK button in the message box. The TDC Emulators Service will normally stop within 10 additional seconds. This can be verified by pressing the Ctrl, Alt and Del keys simultaneously and invoking the Task Manager. From the Task Manager, view the Process tab and verify that the <code>tdcaserver.exe</code> process is no longer in the active process list. See Figure 4.2-2.

Continued on next page

4.2.1 Install to CLM on TPS APP, Continued

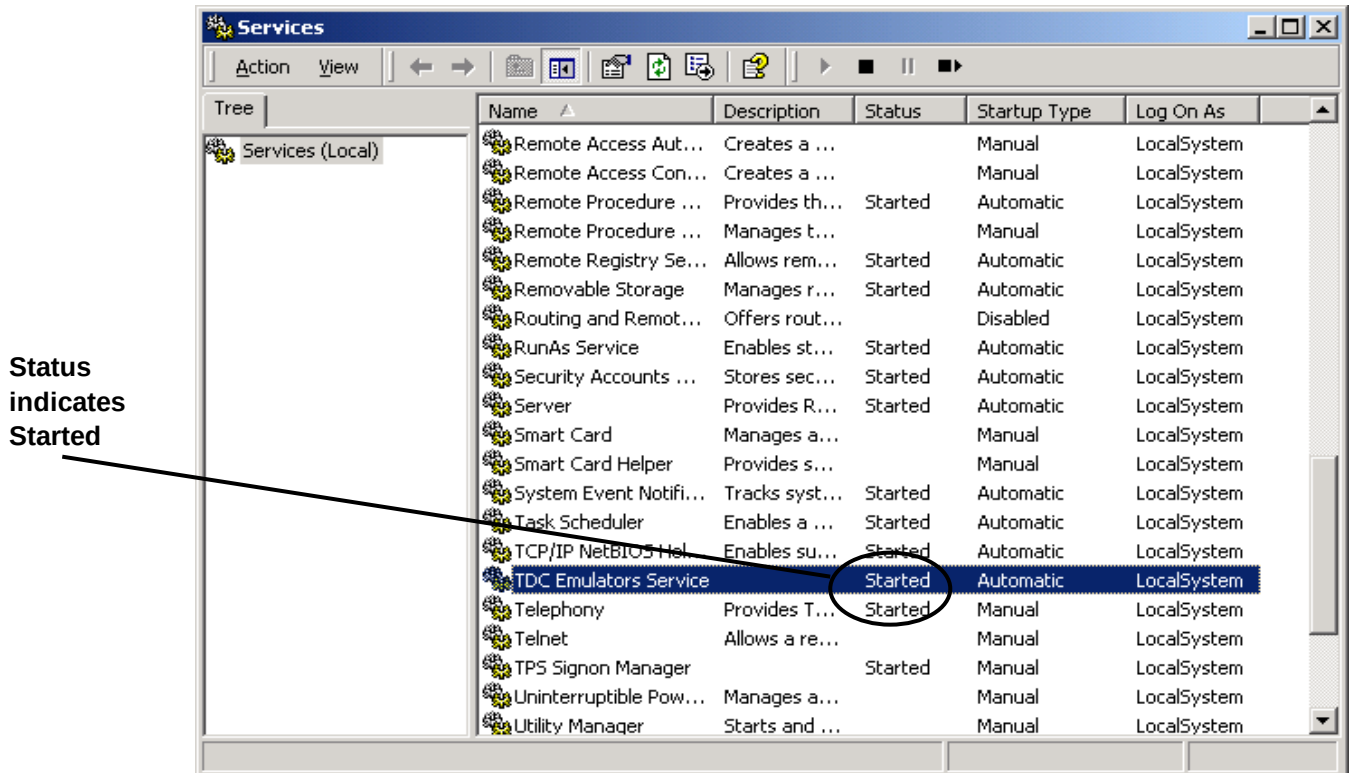


Figure 4.2.1-1 TDC Emulators Service Started

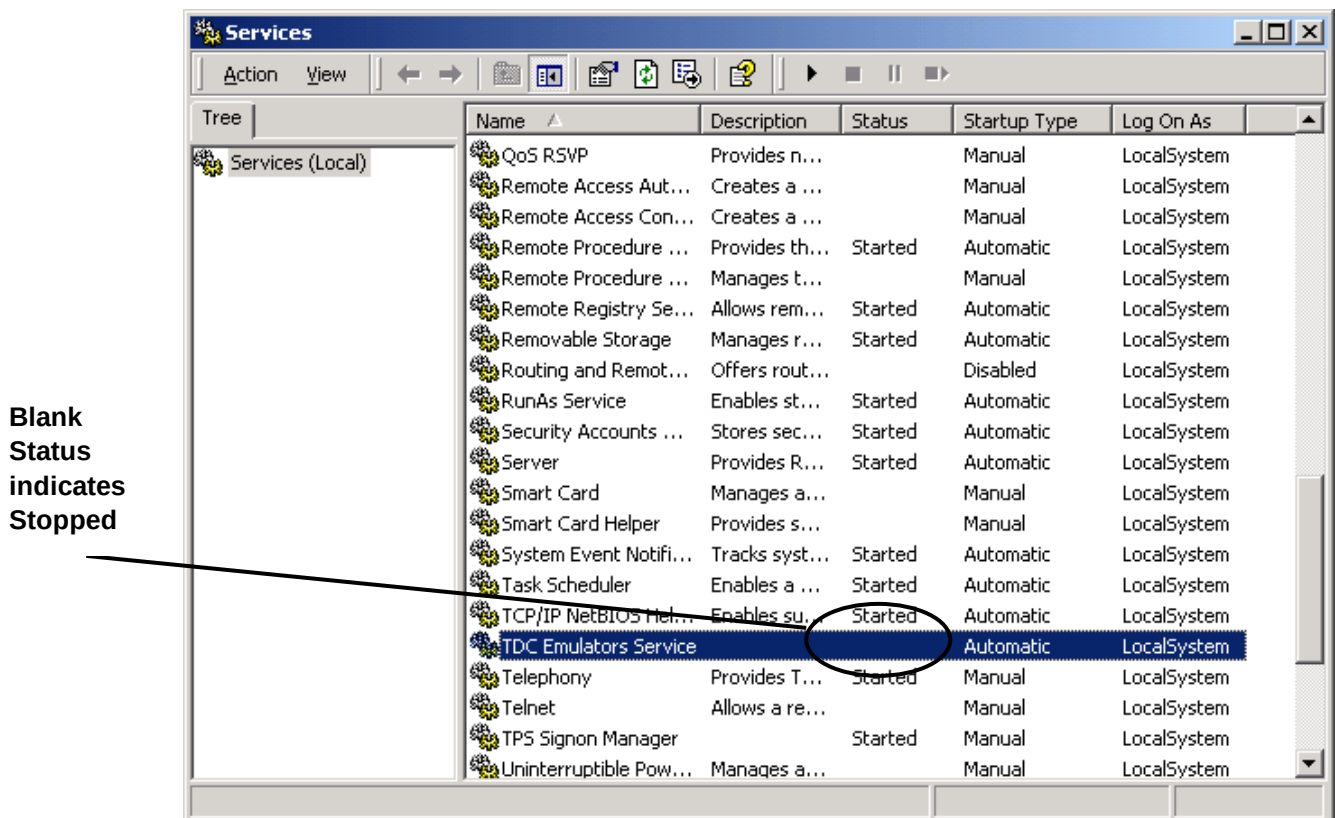


Figure 4.2.1-2 TDC Emulators Service Stopped

Continued on next page

4.2.1 Install to CLM on TPS APP, Continued

Install Modbus Application to Windows

Install the CLM/Modbus Application on the APP.

Step	Action
1	On the TPS APP (Windows side of the APP/CLM), log onto Windows as Administrator.
2	Insert the APP/CLM Modbus distribution media into a drive.
3	View the distribution media in the drive using Windows Explorer. From Windows Explorer, launch the CLM Modbus Install.exe program from the \TPS directory on the distribution media.
4	Follow the instructions for installation. It is recommended that any defaults be selected
5	The actual connection through the COM port to the Modbus network is not established until the CLM Port for the Modbus is placed In-Service.

Continued on next page

4.2.1 Install to CLM on TPS APP, Continued

Define CLMI in the Configuration Utility

First, the virtual CLMI must be defined to the APP, with regard to chassis position and logical board number. This is done through the TPS Configuration Utility, using the CLM option.

Step	Action
1	Determine the logical board number to be used, 1 or 2.
2	Determine the virtual chassis slot position for the CLMI board. This can be 2 through 4. (Position 1 is occupied by the LCNP4 card on the APP)
3	From the Start menu, select Programs, Honeywell TPS, and Configuration Utility.
4	From the drop down menu, select Configure and CLM.
5	Select the Virtual Board 1 or Virtual Board 2 tab, depending on the board number you are working with.
6	Select the check box for the CLMI.
7	Select the drop down menu on the CLMI entry and pick the +Warning(EW) error control.
8	Select the radio button for the virtual chassis slot position.
9	Select the Apply button and OK to exit the CLM Configuration Utility.
10	Select the check box for Application 1.
11	Select the drop down menu for Application 1 and select Modbus.
12	Select the appropriate Ports box(es) for each communication port required.
13	Example: If you were to configure a multiple-application virtual CLMI 1 in logical chassis slot position 3, with Modbus communications through port 2, the CLM Configuration page might look like Figure 4.2-3.

Continued on next page

4.2.1 Install to CLM on TPS APP, Continued

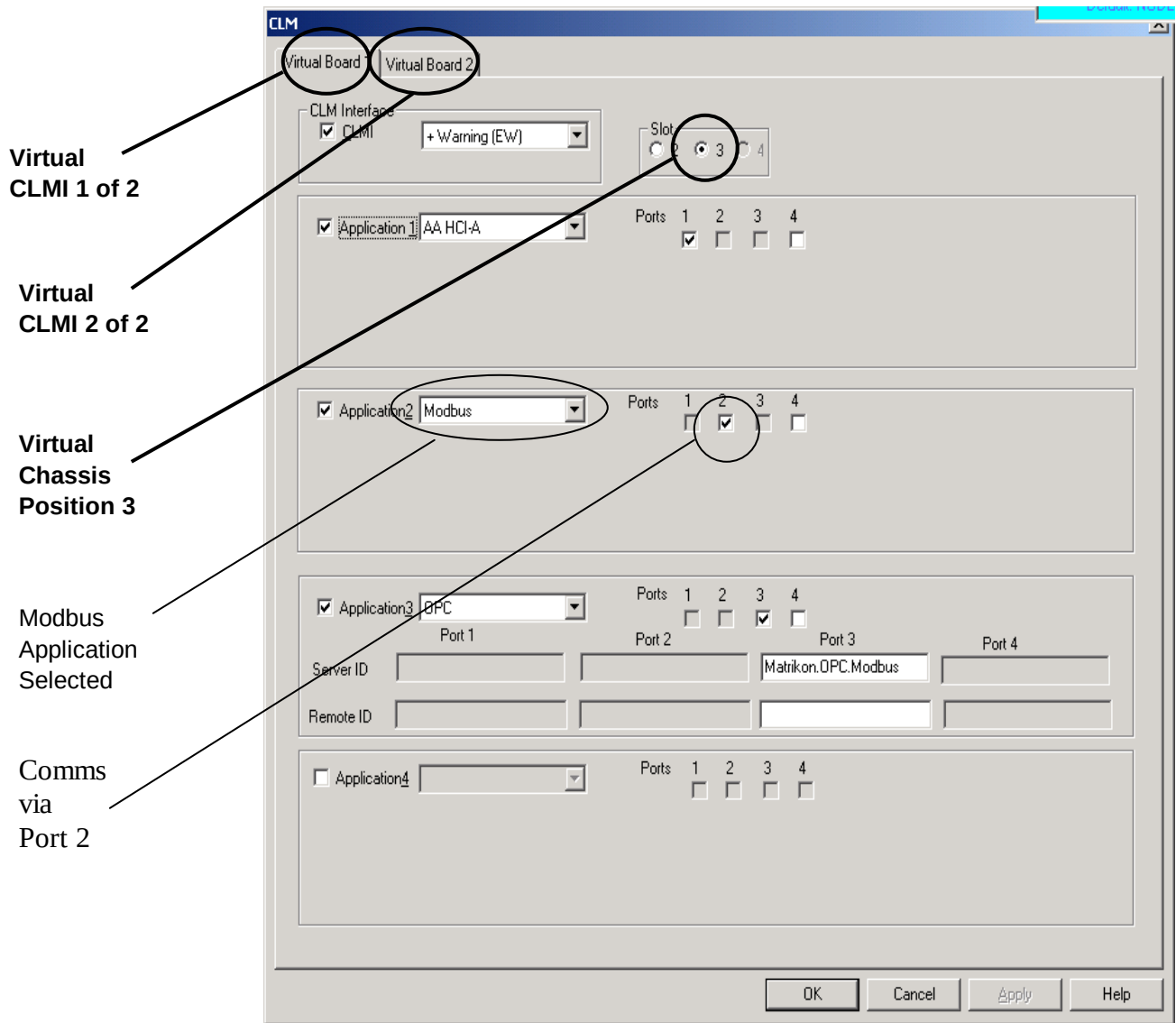


Figure 4.2-3 CLM Configuration Page Example – Modbus on Port 2

4.2.1 Install to CLM on TPS APP, Continued

Start TDC Emulators Service

The TDC Emulators Service simulates the CLMI and CCP boards found in a traditional CLM. Before the TPN personality can be loaded, the TDC Emulators Service must be started and running.

Important! Anytime the CLM configuration is changed from the CLM page of the TPS Configuration Utility, the TDC Emulators Service must be stopped and re-started before the changes will take effect.

Step	Action
1	On the TPS APP (Windows side of the APP/CLM), log onto Windows as Administrator.
2	From the Start menu select Settings/Control Panel/Administrative Tools.
3	Select Services and in the Services window, scroll down until the TDC Emulators Service appears. See Figure 4.2-2 previously.

Note: The Startup option on the TDC Emulators Service should be set to Automatic so that it will start up whenever the APP/CLM node is restarted.

Step	Action
4	Select the TDC Emulators Service entry.
5	From the drop-down menu, select Action, then Properties.
6	If the Startup Type is not Automatic, then select the drop-down menu and change it to Automatic. See Figure 4.2-4 below.
7	Select OK to accept the changes.

Continued on next page

4.2.1 Install to CLM on TPS APP, Continued

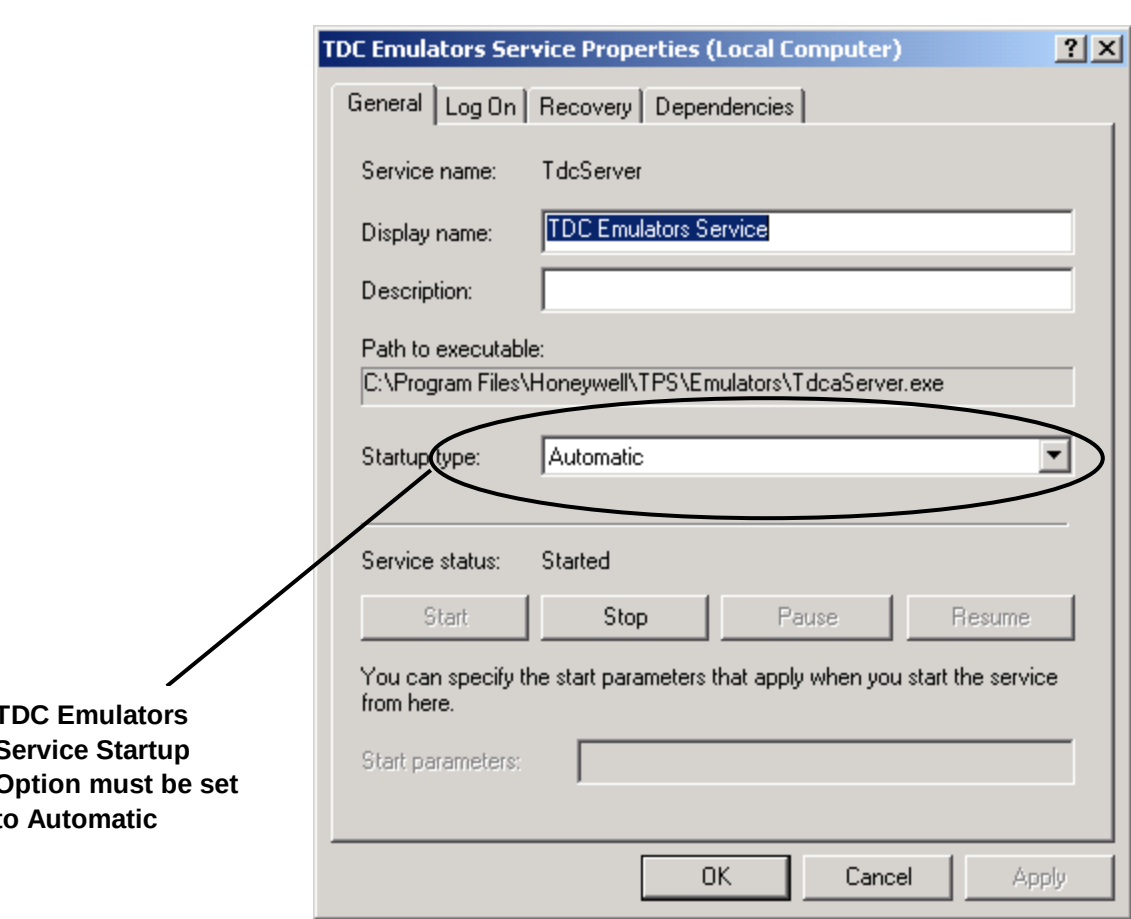


Figure 4.2-4 Automatic Startup option on TDC Emulators Service

2.3.17	If the Status for TDC Emulators Service is Stopped, select the TDC Emulators Service entry. From the drop-down menu, select Action, then the Start button.
2.3.18	A message box appears indicating that the service is being started. This will take about 10-15 seconds. The Services display should appear as in Figure 4.2-1 previously.

The next step This concludes part 1, the installation to Windows.

The next step is to install the LCN side of the CLM/Modbus application.

Proceed to Section 4.3.

4.2.2 Install to CLM on Experion APP

Important! This section is only applicable for users who will be using the Experion node on Windows 2008 operating system (32-bit and 64-bit variants).

Shutdown TDC Emulators Service

In the Windows environment, the Honeywell TDC Emulators Service simulates the peripheral boards that exist in a traditional TPN node chassis. In the APP/CLM, this service simulates the CLMI and CCP boards found in a traditional CLM. Before installation of a APP/CLM Application, this service should be stopped.

Step	Action
1	On the Experion APP (Windows side of the APP/CLM), log onto Windows as Administrator.
2	From the Start menu select Control Panel /System and Security/Administrative Tools.
3	Select Services and in the Services window, scroll down until the TDC Emulators Service appears. See Figure 4.2.2-1 below.
4	If the Status for TDC Emulators Service is Started, select the TDC Emulators Service entry. From the drop-down menu, select Action, then the Stop button.
5	A message box appears indicating that the service is being stopped. This will take about 30 seconds. Note: Occasionally, the TDC Emulators Service does not stop in the required time and an Error Message box appears with the message “<i>Could not stop TDC Emulators Service service</i>”. When this happens, press the OK button in the message box. The TDC Emulators Service will normally stop within 10 additional seconds. This can be verified by pressing the Ctrl, Alt and Del keys simultaneously and invoking the Task Manager. From the Task Manager, view the Process tab and verify that the <code>tdcaserver.exe</code> process is no longer in the active process list. See Figure 4.2.2-2.

Continued on next page

4.2.2 Install to CLM on Experion APP, Continued

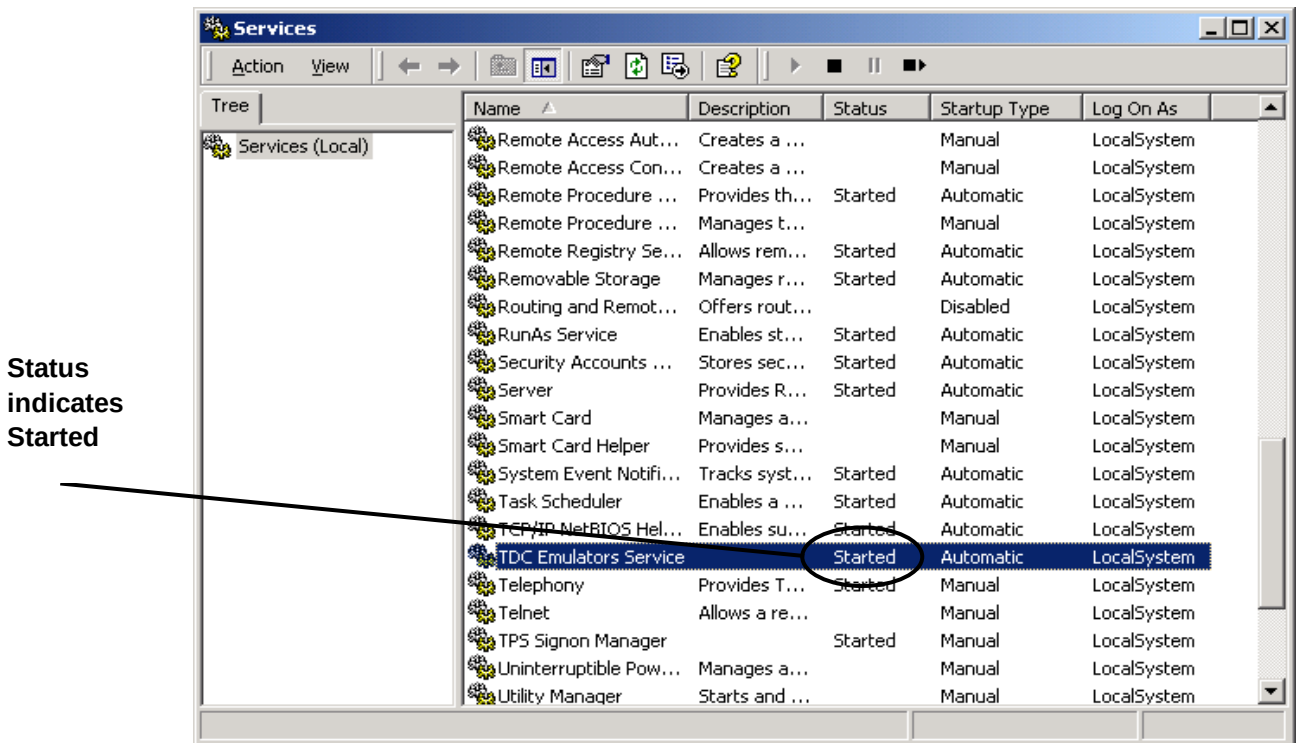


Figure 4.2.2-1 TDC Emulators Service Started

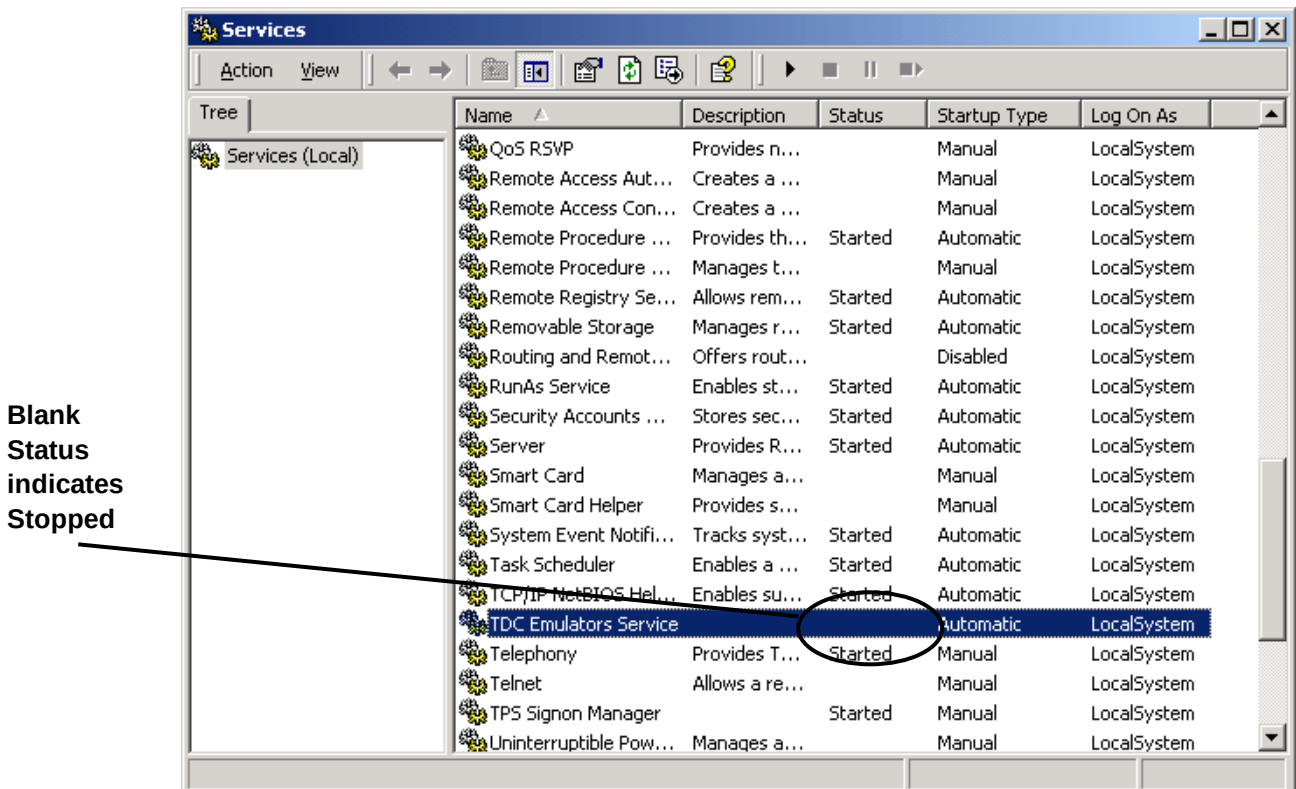


Figure 4.2.2-2 TDC Emulators Service Stopped

Continued on next page

4.2.2 Install to CLM on Experion APP, Continued

Install Modbus Application to Windows

Install the CLM/Modbus Application on the APP.

Step	Action
1	On the Experion APP (Windows side of the APP/CLM), log onto Windows as Administrator. Note: For configuration, you can either use either the System administrator account or Product administrator account.
2	Insert the APP/CLM Modbus distribution media into a drive.
3	View the distribution media in the drive using Windows Explorer. From Windows Explorer, launch the CLM_ Modbus_ Install.msi program from the \Honeywell Experion PKS directory on the distribution media.
4	Follow the on-screen instructions for installation. It is recommended that any defaults be selected
5	The actual connection through the COM port to the Modbus network is not established until the CLM Port for the Modbus is placed In-Service.

Continued on next page

4.2.2 Install to CLM on Experion APP, Continued

Define CLMI in the Configuration Utility

First, the virtual CLMI must be defined to the APP, with regard to chassis position and logical board number. This is done through the Experion Configuration Utility, using the CLM option.

Step	Action
1	Determine the logical board number to be used, 1 or 2.
2	Determine the virtual chassis slot position for the CLMI board. This can be 2 through 4. (Position 1 is occupied by the LCNP4 card on the APP)
3	From the Start menu, select Programs, Honeywell Experion PKS, and Configuration Utility.
4	From the drop down menu, select Configure and CLM.
5	Select the Virtual Board 1 or Virtual Board 2 tab, depending on the board number you are working with.
6	Select the check box for the CLMI.
7	Select the drop down menu on the CLMI entry and pick the +Warning(EW) error control.
8	Select the radio button for the virtual chassis slot position.
9	Select the Apply button and OK to exit the CLM Configuration Utility.
10	Select the check box for Application 1.
11	Select the drop down menu for Application 1 and select Modbus.
12	Select the appropriate Ports box(es) for each communication port required.
13	Example: If you were to configure a multiple-application virtual CLMI 1 in logical chassis slot position 3, with Modbus communications through port 2, the CLM Configuration page might look like Figure 4.2.2-3.

Continued on next page

4.2.2 Install to CLM on Experion APP, Continued

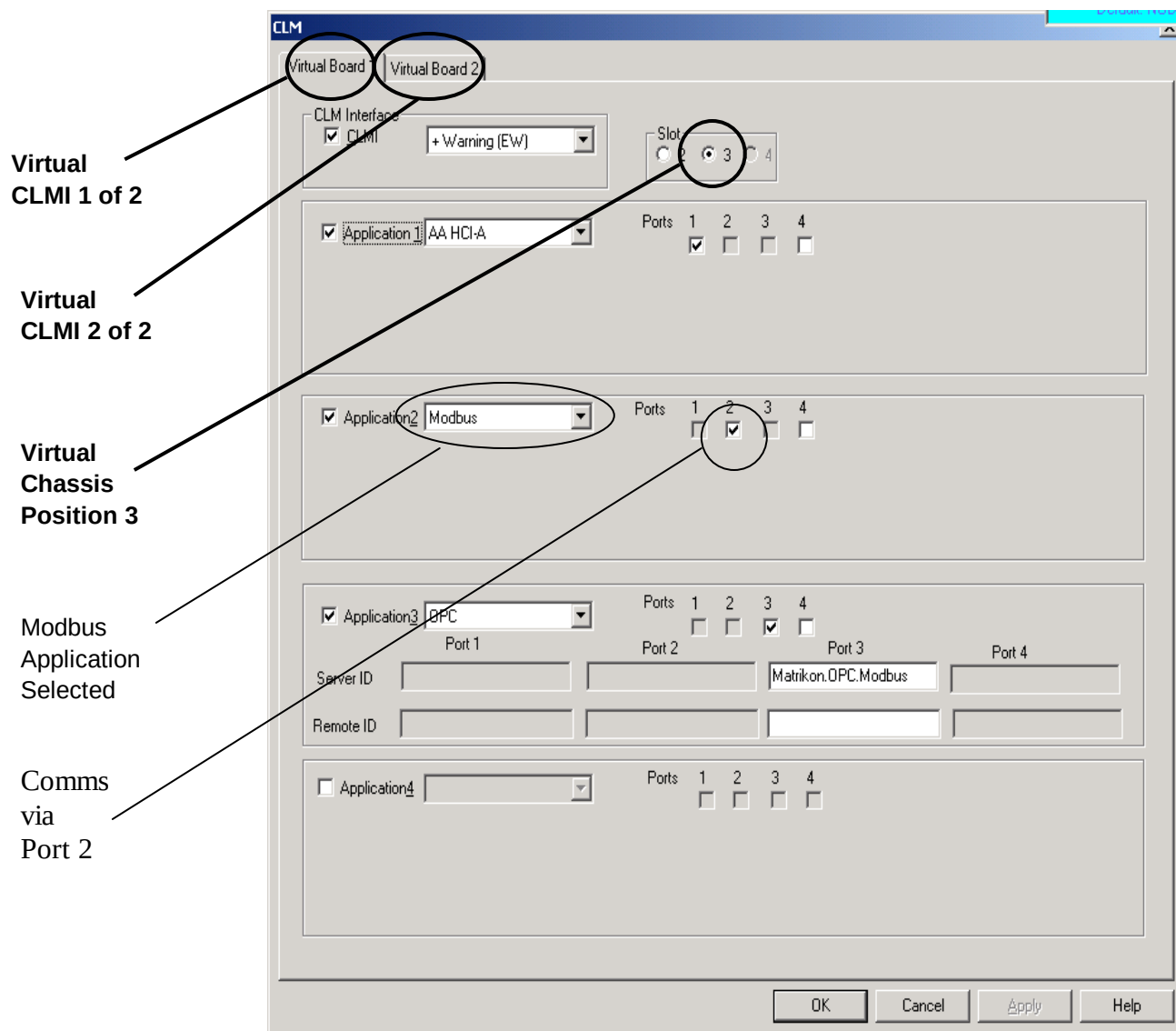


Figure 4.2.2-3 CLM Configuration Page Example – Modbus on Port 2

Continued on next page

4.2.2 Install to CLM on Experion APP, Continued

Start TDC Emulators Service

The TDC Emulators Service simulates the CLMI and CCP boards found in a traditional CLM. Before the TPN personality can be loaded, the TDC Emulators Service must be started and running.

Important! Anytime the CLM configuration is changed from the CLM page of the Experion Configuration Utility, the TDC Emulators Service must be stopped and re-started before the changes will take effect.

Step	Action
1	On the Experion APP (Windows side of the APP/CLM), log onto Windows as Administrator.
2	From the Start menu select Control Panel /System and Security/Administrative Tools.
3	Select Services and in the Services window, scroll down until the TDC Emulators Service appears. See Figure 4.2.2-2 previously.

Note: The Startup option on the TDC Emulators Service should be set to Automatic so that it will start up whenever the APP/CLM node is restarted.

Step	Action
4	Select the TDC Emulators Service entry.
5	From the drop-down menu, select Action, then Properties.
6	If the Startup Type is not Automatic, then select the drop-down menu and change it to Automatic. See Figure 4.2.2-4 below.
7	Select OK to accept the changes.

Continued on next page

4.2.2 Install to CLM on Experion APP, Continued

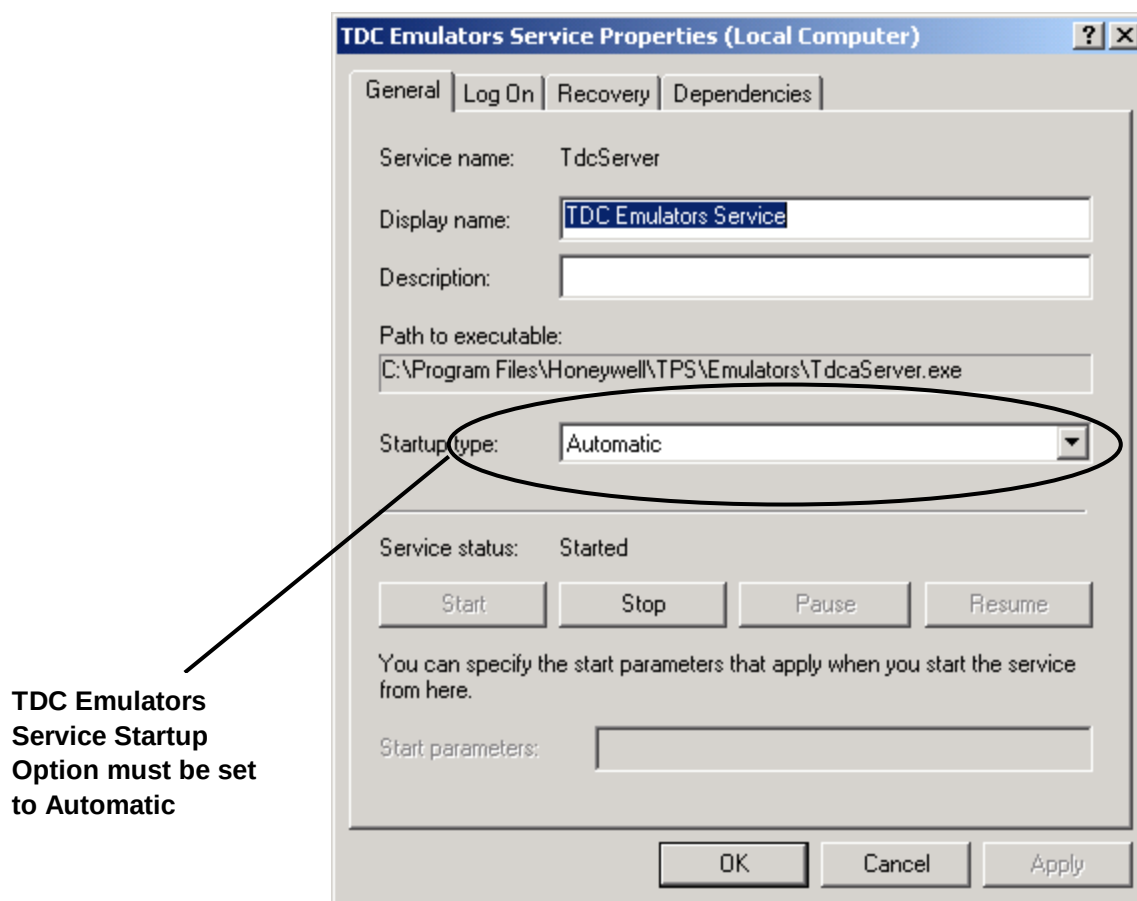


Figure 4.2.2-4 Automatic Startup option on TDC Emulators Service

2.3.17	If the Status for TDC Emulators Service is Stopped, select the TDC Emulators Service entry. From the drop-down menu, select Action, then the Start button.
2.3.18	A message box appears indicating that the service is being started. This will take about 10-15 seconds. The Services display should appear as in Figure 4.2.2-1 previously.

The next step

This concludes part 1, the installation to Windows.

The next step is to install the LCN side of the CLM/Modbus application.

Proceed to Section 4.3.

4.3 Install to LCN

Overview

This section details the installation procedure that applies to the files required by the LCN side of the CLM.

This procedure must be executed for both APP/CLM and CLM.

STOP! The following actions **MUST** be completed prior to proceeding with this next part of the installation!

- The installation of the CLM Platform software must be completed before continuing with this part of the installation.
- For APP/CLM, Section 4.2 must be completed before continuing with this part of the installation.

Prior to Installation

Steps to be performed prior to part 2 of the installation (LCN).

Step	Action
1	From the Engineering Personality Main Menu of the Native Window on a User station, select Support Utilities, then select Modify Volume Paths
2	Verify the correct path for the CL SOURCE/OBJ. We suggest that you use NET>UCLM.
3	Verify the path for the CL CUSTOM GDF is NET>CDSG (or your system repository directory). We recommend that only one network directory be used, and not changed.
4	Set the CL PARAM LIST to the same path as the CL SOURCE/OBJ defined in step 2. This information is used when certain CDS modules are compiled.
5	Set the USER DEFLT PATH to the same path name as the CL SOURCE/OBJ defined in step 2. This information is used when the supplied CDS modules are compiled and when the executable module link and unlink directives are executed.
6	Review the files in the product release which will be copied to your system to assure no conflicts (duplicate names) in file naming with files already on your system. This is especially critical for CL and CDS files which may be overwritten. Ref. Table 3.3-1.
7	Before the install procedure is executed, the paths must be identified and the volume of &CUS and &CLX must exist on the HM. Normally, these directories are created during the NCF configuration of the AM.

4.3.1 Move Files to LCN

STOP!

In preparation for executing the LCN installation procedure, the following items should be reviewed:

- Read the TPS/TPN Customer Release Guide for information that applies to your installation including HM sizing requirements, special instructions, and reinstallation/migration information
 - Scan the file - M_INST.EC - provided on the supplied media for an overview of the LCN installation process.
-

Installation EC Script Actions

The installation file (M_INST.EC) performs the following actions:

- checks for a previous installation of the CLM/Modbus application package.
- determines the TPN LCN release for loading of the specific release dependent files to the HM. Some files to be loaded are independent of the TPN release, while other files must be loaded or compiled only for a specific release.
- requires confirmation of load path (directories) setup - Section 4.3.
- copies required files from the supplied media to the designated HM directories.
- compiles supplied Custom Data Segments (CDS's) required for package operation.
- creates a file, M_INST.LS, of the EC script execution for verification of successful installation (no errors listed) and for future reference.

On completion of the EC script, the package files will be resident on the destination HM and installation procedures may continue as described in the following sections.

Run the EC Script

In the Command function within the Engineering Personality, enter the following command to begin the install procedure:

EC \$Fn>&MB>M_INST.EC \$Fn

where \$Fn = the logical device identification (i.e. \$F1)

Follow the prompts output by the install execution and enter the required information to continue through the procedure.

Important! Scan the execution listing file – M_INST.LS. If any errors exist in this file, they must be corrected before continuing.

4.3.2 Compile the LCN Pictures

Compile pictures

Pictures included in the package must now be compiled. There is only one picture required for the Modbus application, CLMMODB0.DS. This picture is used to allow display and output of strings for the message point.

Steps to Compile Modbus application pictures

Step	Action
1	Set the USER DEFLT PATH to the same path as your destination path for displays.
2	The custom display for the application needs to be compiled. To accomplish this, the picture in the following list needs to be read and compiled. • CLMMODB0
3	In the Engineering Personality, invoke the Picture Editor.
4	Enter the commands: R CLMMODB0 LD CLMDDB (from Platform installation)
5	Enter the command: COM

The Next Step

All files required for the Modbus application installation are now moved and/or compiled. The next step is to begin configuration of the CLM/Modbus interface in Network Memory (defined in Section 2.1).

4.4 Configure the CLM Loader Directive File

Modbus protocol layer

The CLM Platform provides the lower level functionality to support the protocol layer on top of the platform. The protocol layer, developed by Honeywell, will include the software and other pieces required to communicate with the external devices. This protocol layer is the Modbus interface. This step defines the configuration information required by the platform to identify the hardware and software used to accomplish communications with the external devices.

CLMI loader directive file

Since the interface board is capable of supporting different protocols on each port, or Modbus protocol across several of the ports, this configuration layout must be defined so that the appropriate protocol driver set is loaded to the proper ports. This is accomplished by the CLM loader directive file. This file is queried by the CLM process software for loading directions every time the one or both of the interface boards is placed logically in service.

Each CLM node must have its own CLMI load directive file. This configuration file is an ASCII text file with the following naming convention:

- The name is the literal CLMI followed by three digit CLM physical node number (001-096). Redundant node pairs use the lower pair node number (i.e. redundant CLM 33/34 uses CLMI033.ZC).
- The file extension is the literal ZC.
- The file default location is the NET>&CUS> directory.

For example, NET>&CUS>CLMI024.ZC is the CLM configuration file name for physical node 24.

Note: Although the name implies the use of the CLMI board, this file applies as well to the CCP and UCIM board sets.

Two versions of the loader directive file -

The APP/CLM (APP) and CLM (AM) have different loader directive requirements.

Go to the Appropriate Section

4.4.1 Describes the APP/CLM (APP) loader directives

4.4.2 Describes the CLM (AM) loader directives

Use the appropriate section for your installation.

4.4.1 Configure the APP/CLM (APP) Loader Directive File

Format

The CLMI load directive file creation is a part of the CLM Platform installation procedure. This implementation requires a modification of that directive file.

The following directives are used in the CLMI loader directive file on the APP/CLM:

```
CLMI <board #>, <slot #>  
END
```

Where

board # = the logical board number (1 or 2) to be defined within the CLMI NDT entry.

Slot # = the chassis slot number where the board would be located if this was a physical five card chassis. Slots 2 through 5 may be used. Default = 3.

Note: Unlike the AM version of the CLM, the APP/CLM requires only a CLMI directive and an END directive in the CLM Loader Directive file.

Specific information on the configuration of APP/CLM applications is entered using the Honeywell TPS Configuration Utility from the Windows side of the APP.

Edit your configuration file

Using the LCN Editor, or other appropriate editing tool, create the Loader Directive File (.ZC) and enter the two fields above for each board that is emulated in the APP/CLM (max 2).

This file must be saved and reside on the History Module at the required directory (Default = NET>&CUS).

Examples

Configure a single virtual CLMI in logical chassis slot 3:

```
CLMI 1,3  
END
```

Configure two virtual CLMIs in logical chassis slot 2 and 3:

```
CLMI 1,2  
END  
CLMI 2,3  
END
```

STOP!

Skip the next Section 4.4.2! This section deals with the Loader Directive File for the AM-based CLM only!

4.4.2 Configure the CLM (AM) Load Directive File

Format

The CLMI load directive file creation is a part of the CLM Platform installation procedure. This implementation requires a modification of that directive file.

The form of the configuration directives to define the CLM environment is:

```
CLMI <board #>, <slot #>
LOAD CCP.S                (CLMI boards use CLMI.S)
RUN 7F001100              (CLMI boards use RUN 41100)
LOAD MODB.S               (Application file - Modbus)
INIT 0, <port map>
END
```

Where

- board # = the logical board number (1 or 2) to be defined within the CLMI NDT entry.
- slot # = the chassis slot number where the board is physically located in LCN node chassis. Slots 2 through 5 may be used.

Interface Board slot number

Visually, or using the Maintenance Personality on a user station, determine the CLM interface board slot numbers for each CLM node.

Port Map

The port map is an eight-digit number. The first four digits correspond to the four board ports. *The remaining four digits are reserved for future use and must be zero.* Enter a one in the corresponding port map position to assign a port driver to a specific port. Enter a zero if the port is not used for this application.

Format: INIT 0,ABCD0000

Where

- A = Port 1 Application: 1 = Configured, 0 = Not Configured
- B = Port 2 Application: 1 = Configured, 0 = Not Configured
- C = Port 3 Application: 1 = Configured, 0 = Not Configured
- D = Port 4 Application: 1 = Configured, 0 = Not Configured

Continued on next page

4.4.2 Configure the CLM (AM) Load Directive File,

Continued

Edit your configuration file

Next, add configuration information to the CLMI configuration file that was installed as part of the CLM Platform software (for example, CLM node 37 would be CLMI037.ZC).

Edit Your Configuration File

Step	Action
1	Set the current path name to NET>&CUS.
2	Using the text file editor, call up the text file NET>&CUS>CLMI0nn.ZC, where 0nn is the CLM physical node number. For example, CLMI024.ZC is the CLMI configuration file for CLM node 24.
3	The first 6 directives are always used. The following example configures the first board and first two ports for the Modbus application on a CCP interface board residing in physical slot 5 of the chassis. CLMI 1,5 LOAD CCP.S RUN 7F001100
	LOAD MODB.S INIT 0,11000000
	END
4	If two boards are used, then add the following directives after the first END to configure the second board and first port for the Modbus application on a CCP board residing in physical slot 4 of the chassis. CLMI 2,4 LOAD CCP.S RUN 7F001100
	LOAD MODB.S INIT 0,10000000
	END

The next step

This concludes the Modbus Installation Procedure.

The next step is to begin the configuration process. First, you must define the Modbus network to the CLM by defining the Network Memory NDTs.

Please continue now with Section 5 of this manual.

Section 5 – Modbus Interface Design

5.1 Overview

Introduction

The Modbus Interface Design consists of the following tasks.

- Network Design and Configuration
 - Network Status and Performance Displays
 - Organization of PITs and PITGs
-

Reference Documents

Obtain and understand the following documents.

- *CLM Platform Implementation Manual*
 - Modbus Application Manuals
 - An overview diagram of your Modbus installation: Identify the address of each Modbus device in your installation.
 - A listing of the Modbus coils and registers to be imaged by AM points in your installation.
-

Modbus Configuration Overview

The following table is an outline of the steps used in the design and configuration of a CLM interface to a Modbus device or network.

Continued on next page

5.1 Overview, Continued

Addressable components

From the CLM view, a Modbus network is defined by using four addressable components. These components are listed below from the CLM level down to the lowest Modbus device component (i.e. register).

Modbus Addressable Components

Component	Description
NDT Address Components	
CLMI	CLMI or CCP Board; 1 or 2 Boards per CLM For APP/CLM, remember that the board is emulated and does not physically exist..
PORT	Port; 1 to 4 Ports per CLMI/CCP
DEVICE	Modbus Device (address range 1 – 247)
Address entered in PITE	
POINT.PARAMETER	Register or coil point number

NDT Entries

NDT entries belong to a hierarchy, and must be built in the following sequence:

NDT entries for CLMI, then PORT then DEVICE.

PITG entries (optional)

PIT entries

Note that the PIT entries require the use of the name of a device NDT. All of the above entries are built before you build the PITE entries for individual points (discussed in Section 6 of this manual).

5.2 Network (NDT) Design and Configuration

CLM Configuration

Overview: Steps for Modbus network (NDT) configuration

Step	Activity												
1	Identify the Template Definition Files (TDFs) for each type of addressable component in network. The TDFs are provided by Honeywell and will be used in building the Network configuration. The CLMI TDF is detailed in the <i>CLM Platform Implementation Manual</i>. The table following lists the three basic templates required to define the Modbus network. <table><tr><th>Template Name (ZT)</th><th>Description</th></tr><tr><td>CLMI</td><td>CLMI or CCP Interface Board (from CLM Platform)</td></tr><tr><td>M_PORT</td><td>CLMI Port (up to 4)</td></tr><tr><td>M_DEVICE</td><td>Modbus Physical Device</td></tr></table>	Template Name (ZT)	Description	CLMI	CLMI or CCP Interface Board (from CLM Platform)	M_PORT	CLMI Port (up to 4)	M_DEVICE	Modbus Physical Device				
Template Name (ZT)	Description												
CLMI	CLMI or CCP Interface Board (from CLM Platform)												
M_PORT	CLMI Port (up to 4)												
M_DEVICE	Modbus Physical Device												
2	Each template file, or network component, has an associated Status Detail Definition file (.ZS). This file is used by the CLM graphic package to display specific detailed data about that device.												
3	Identify each addressable network component. The number of addressable network components will vary from site to site; however, the table below shows that all Modbus networks will have the following types that match respectively with the template files listed in the above table. <table><tr><th>Component</th><th>Description</th><th>Range</th></tr><tr><td>CLMI</td><td>CLMI or CCP Interface Board</td><td>1-2</td></tr><tr><td>PORT</td><td>Communication Port on the board</td><td>1-4</td></tr><tr><td>Device</td><td>One for each Modbus device Note: Device number 0 is used for Modbus broadcast messages.</td><td>1-247</td></tr></table>	Component	Description	Range	CLMI	CLMI or CCP Interface Board	1-2	PORT	Communication Port on the board	1-4	Device	One for each Modbus device Note: Device number 0 is used for Modbus broadcast messages.	1-247
Component	Description	Range											
CLMI	CLMI or CCP Interface Board	1-2											
PORT	Communication Port on the board	1-4											
Device	One for each Modbus device Note: Device number 0 is used for Modbus broadcast messages.	1-247											

Continued on next page

5.2 Network (NDT) Design and Configuration, Continued

Overview: Steps for Modbus network (NDT) configuration, continued

Step	Activity
4	Step 4 and the creation of a BIF is optional. You can configure the network on-line one entry at a time by using the interactive build feature of the CLM Configuration Builder. Create a BIF to include one NDT entry for each of the identified addressable network components. • The BIF may be created using the file editor, or other text file editor, and transferred to the HM directory of your choice (default is NET>&CUS) for use. • Names of the prototype BIFs included by Honeywell are listed in Table 3.3-1 ('Device' file extension .ZI). • Typical BIF samples for the CLMI, Port and Device are included in this section. • Copy the appropriate BIF file, and then complete each definition within the BIF, to create the BIF NDT entry for each addressable network component.
5	Using the CLM Configuration Builder, build the NDT entries on the target CLM node • Instructions for using the CLM Configuration Builder are detailed in the <i>CLM Platform Implementation Manual</i>.

User modified Device NDT files

You may want to customize your device NDT templates so that the various options required, which are predetermined for a given Modbus vendor device, are permanently entered in the template. By *carefully* creating a custom template, you avoid reentering these parameters for each device. When you create custom templates, be sure to:

- Assign a unique name (The name M_DEVICE is reserved.)
- Make a .ZT, & .ZI, file for all created templates and a .ZS file for each new device type template.
- Enter the correct template name on the ENTRY line.

Important ! The template name on the ENTRY line and the file name (*.ZT) must be IDENTICAL.

5.2.1 CLMI – Define and Configure the NDT

Introduction

As stated earlier, Modbus requires the following Network Definition Table (NDT) entries to be defined. Builder Input File forms are included with the release and may be used to configure each NDT. Otherwise, each entry may be configured singly on-line with the Configuration Builder. Refer to the *CLM Platform Implementation Manual*. All network devices must be defined.

This section defines the CLMI or CCP interface boards on your system.

Template Name	Description
CLMI.ZT	CLMI Interface Board (Platform)
M_PORT.ZT	CLMI Port
M_DEVICE.ZT	Modbus Physical Device

Define your boards

One CLMI NDT must be built for each board to be configured within the CLM. The board(s) may be numbered as 1 or 2.

Using the Configuration Builder, or the appropriate ZI file, and the BIF field definitions below, define the CLMI or CCP board(s).

CLMI.ZI

CLMI.ZI - Typical BIF for the CLMI/CCP Interface Board

```
ENTRY  = CLMI
NAME   =
A1_CLMI =
FAILOVER =
END
```

CLMI BIF Fields

BIF Fields Identifying the CLMI/CCP Interface for CLM

Step	Field	Action
1	NAME	Enter a text name for the CLMI board component, such as CLMI_1.
2	A1_CLMI	Enter the logical board number: 1 is the default.
3	FAILOVER	Must be N for a APP/CLM On redundant CLMs, Enter Y to initiate automatic switchover of the CLM node when the CLMI board fails; leave the entry at the default of N if the CLM is non-redundant or a APP/CLM.

5.2.2 Port – Define and Configure the NDT

Introduction

As stated earlier, Modbus requires the following Network Definition Table (NDT) entries to be defined. Builder Input File forms are included with the release and may be used to configure each NDT. Otherwise, each entry may be configured singly on-line with the Configuration Builder. Refer to the *CLM Platform Implementation Manual*. All network devices must be defined.

This section defines the application ports on the configured boards that will send/receive data from the Modbus network(s).

Template Name	Description
CLMI.ZT	CLMI Interface Board (Platform)
M_PORT.ZT	CLMI Port
M_DEVICE.ZT	Modbus Physical Device

Define your ports

One M_PORT NDT must be built for each port on the configured board(s) that will be configured for Modbus communications. The port(s) must be numbered from 1 to 4. *The ports are numbered from left to right as you look directly at the back of the CLM chassis at the installed IO paddleboard of the CLMI or CCP board.*

Using the Configuration Builder, or the appropriate ZI file, and the BIF field definitions below, define the port(s) to be used on the board(s).

M_PORT.ZI

M_PORT.ZI - Typical BIF for the Modbus Port

```
ENTRY  = M_PORT
NAME   =
A1_CLMI =
A2_PORT =
FAILOVER =
FAIL_TM =
*Baud (0=300 1=1200 2=2400 3=4800 4=9600 5=19200 6=38400)
BAUD   =
SBITS  =
PARITY  =      * (NONE=0,ODD=1,EVEN=2)
RETRY   =
TIMEOUT =
DIAG_CYC =
REG_GAP =
MODEM   =
END
```

Continued on next page

5.2.2 Port – Define and Configure the NDT, Continued

M_PORT BIF Fields

BIF Fields Identifying the Modbus Ports

Step	Field	Action																
1	ENTRY	M_PORT																
2	NAME	Enter a text name identifier for the port component, such as PORT_1.																
3	A1_CLMI	Enter the logical CLMI/CCP board: 1 or 2. (Default 1)																
4	A2_PORT	Enter the port number to be used: 1 to 4 (Default 1)																
5	FAILOVER	Must be N for a APP/CLM On redundant CLMs, Enter Y to initiate automatic switchover of the CLM node when the port fails. Leave the entry at the default of N if the CLM is non-redundant or a APP/CLM. Important! If the FAILOVER option is selected (Y), then the FAIL_TM field in the template (.ZT) file must be made user-visible and a time value entered (See <i>CLM Platform Implementation Manual</i>).																
6	FAIL_TM (FAILOVER=Y)	Only if FAILOVER = Y (Default = -1) Enter the time value in seconds or minutes that must elapse when the port fails before the CLM will initiate an automatic switchover. -1 = no switchover, 0 = immediate failover																
7	BAUD	Enter the baud rate code for this communication port. Valid codes and their meanings are: <table border="1"><thead><tr><th>Code</th><th>Baud Rate</th></tr></thead><tbody><tr><td>0</td><td>300</td></tr><tr><td>1</td><td>1,200</td></tr><tr><td>2</td><td>2,400</td></tr><tr><td>3</td><td>4,800</td></tr><tr><td>4</td><td>9,600 (Default)</td></tr><tr><td>5</td><td>19200</td></tr><tr><td>6</td><td>38400 *</td></tr></tbody></table> * Must have 51305055-100 CCP Paddleboard or be resident in a APP/CLM.	Code	Baud Rate	0	300	1	1,200	2	2,400	3	4,800	4	9,600 (Default)	5	19200	6	38400 *
Code	Baud Rate																	
0	300																	
1	1,200																	
2	2,400																	
3	4,800																	
4	9,600 (Default)																	
5	19200																	
6	38400 *																	
8	SBITS	Enter the number of stop bits, 1 or 2 (Default 1)																

Continued on next page

5.2.2 Port – Define and Configure the NDT, Continued

M_PORT BIF Fields

BIF Fields Identifying the Modbus Ports, continued

Step	Field	Action								
9	PARITY	Enter the parity code. Valid codes and their meanings are: <table><tr><th>Code</th><th>Parity</th></tr><tr><td>0</td><td>None (Default)</td></tr><tr><td>1</td><td>Odd</td></tr><tr><td>2</td><td>Even</td></tr></table>	Code	Parity	0	None (Default)	1	Odd	2	Even
Code	Parity									
0	None (Default)									
1	Odd									
2	Even									
9	RETRY	Enter the RETRY code for this port. This defines the number of retries that will be attempted on a communication failure before aborting the request and marking the device as failed. The default is 2, which means a total of 3 attempts to communicate will be made before aborting the request.								
10	TIMEOUT	Request time-out period. This is the base time-out period (in tenths of a second) with no response from the device that the board will wait before the request is retried. For example, 50 means 5 seconds. (Default 5.)								
11	DIAG_CYC	Diagnostic task cycle (seconds) (Default 300) If diagnostics are configured, and supported by the device, then requests will be sent at this interval. See the M_DEVICE field definitions.								
12	REG_GAP	Address optimization gap. Used when optimizing read requests across a small gap. The number can range from 0 to 123. <i>There is a direct correlation between the baud rate and the amount of data to be retrieved. In some instances, it is more bandwidth efficient to do a block read of a group of data to get only a few as opposed to making several single requests.</i> We suggest smaller gaps at slower baud rates. (Default 40)								
13	MODEM	Enter Y if this port is connected to a modem. or Enter Y if the port is hardware-configured for RS485. (Default is N)								

Continued on next page

5.2.2 Port – Define and Configure the NDT, Continued

Modifying the Port

There may be instances where changes are required to the existing port entry. The impact of changes to fields within the port entry fall into two categories.

- Field changes that immediately affect the port operation are called **dynamic** changes.
- Field changes that require a restart of the CLMI/CCP board are called **potential** changes.

The table below defines each of the fields in the Port NDT as either dynamic or potential. When making changes to the port, keep in mind the impact of these changes.

Field	Impact of change
FAILOVER	Dynamic
FAIL_TM	Dynamic
BAUD	Potential
SBITS	Potential
PARITY	Potential
RETRY	Dynamic
TIMEOUT	Dynamic
DIAG_CYC	Dynamic
REG_GAP	Dynamic
MODEM	Potential

5.2.3 Device – Define and Configure the NDT

Introduction

As stated earlier, Modbus requires the following Network Definition Table (NDT) entries to be defined. Builder Input File forms are included with the release and may be used to configure each NDT. Otherwise, each entry may be configured singly on-line with the Configuration Builder. Refer to the *CLM Platform Implementation Manual*. All network devices must be defined.

This section defines the physical device(s) on the Modbus network that will respond to data requests from the CLM interface board.

Template Name	Description
CLMI.ZT	CLMI Interface Board (Platform)
M_PORT.ZT	CLMI Port
M_DEVICE.ZT	Modbus Physical Device

Define your devices

One M_DEVICE NDT must be built for each device on the Modbus network. Each device must be examined to gather all of the communications characteristics, such as node address, baud rate settings, which function codes are supported, etc.

Note: The CLM is considered to be the Modbus *Master* and all devices configured on the Modbus network are to be configured as Modbus *Slave* devices.

Using the Configuration Builder, or the appropriate ZI file, and the following BIF field definitions, define the device(s) to be configured on each configured port.

Continued on next page

5.2.3 Device – Define and Configure the NDT, Continued

M_DEVICE.ZI

M_DEVICE.ZI - Typical BIF for the Modbus Device

```
ENTRY      = M_DEVICE
NAME       =
A1_CLMI    =
A2_PORT    =
A3_DEV     =
* Diagnostic Option
D00        =
D02        =
D11        =
D12        =
D13        =
D14        =
D15        =
D16        =
D17        =
D18        =
D19        =
FC07       =
* Maximum query and response data
FC01       =
FC02       =
FC03       =
FC04       =
FC05       =
FC06       =
FC15       =
FC16       =
RBE_FC     =
END
```

Continued on next page

5.2.3 Device NDT Builder Input File Format, Continued

M_DEVICE BIF Fields

BIF Fields Identifying the Modbus Devices

Step	Field	Action
1	ENTRY	M_DEVICE
2	NAME	Enter a text name identifier for the Modbus device. The name must be unique within the NDT entries.
3	A1_CLMI	Enter the logical CLMI/CCP board: 1 or 2. (Default 1)
4	A2_PORT	Enter the port number to be used: 1 to 4 (Default 1)
5	A3_DEV	Enter the Modbus address of the device: 1 to 247. (Default 1)
Enable Diagnostic Codes (Enter Y to enable Default is N)		
6	D00	Return Query Data
7	D02	Return Diagnostic Register
8	D11	Return Bus Message Count
9	D12	Return Bus CRC Error Count
10	D13	Return Bus Exception Error Count
11	D14	Return Slave Message Count
12	D15	Return Slave No Response
13	D16	Return Slave NAK Count
14	D17	Return Slave Busy Count
15	D18	Return Bus Character Overrun Count
16	D19	Return Overrun Error Count
17	FC07	Enable Read Exception Status (Function code 07)
Maximum Query and Response Data for the Device (Default is 0) (i.e. maximum number of registers/coils per request)		
18	FC01	Read Coil Status, COILS / query (max 2000)
19	FC02	Read Input Status, INPUTS / query (max 2000)
20	FC03	Read Holding Register, REGISTERS / query (max 125)
21	FC04	Read Input Register, REGISTERS / query (max 125)
22	FC05	Force Coil, COILS / request (max 1)
23	FC06	Load Register, REGISTERS / request (max 1)
24	FC15	Force Multiple Coils, COILS / request (max 800)
25	FC16	Preset Multiple Registers, REGISTERS / request (max 100)
26	RBE_FC	RBE Query Function Code (Future Implementation)

5.3 NDT Status Detail Displays

Introduction

This section is for information only. It defines the status detail display layouts for the CLMI/CCP, Port, and Device level components within the Modbus application.

These displays show dynamic configuration and communications data for the associated NDT device. The displays are refreshed every four seconds by the CLM software. All status detail displays are invoked from the CLM Network Status Display.

The format of each of the status detail displays is defined within text files resident on the HM. The file name matches the NDT template name with a ZS extension. For example, the port status detail display would be M_PORT.ZS.

Note! The M_PORT and M_DEVICE detail status displays can be very useful when establishing communication with the Modbus network. Communications statistics, such as timeouts, retries, CRC errors, Parity errors and such are available dynamically to the user from these displays.

Status Display for CLMI (CLMI.ZS)

```
### - NO. OF UNACKNOWLEDGED FAILED DEVICES ON THE CLMI
### - NO. OF ACKNOWLEDGED FAILED DEVICES ON THE CLMI
```

Average Parameters per Second

	INPUT	OUTPUT
4 Sec.	####	####
5 Min.	####	####

```
##### - NO. OF PPSs [CLM_PPS, CLM_OUT and PPS]
```

Status Display for Port (M_PORT.ZS)

Modbus Release 1 Revision 2

```
### - NO. OF UNACKNOWLEDGED FAILED DEVICES ON THE PORT
### - NO. OF ACKNOWLEDGED FAILED DEVICES ON THE PORT
```

##### - MESSAGES TRANSMITTED	##### - MESSAGE CRC ERRORS
##### - MESSAGES RECEIVED	##### - PROTOCOL ERRORS
##### - NO RESPONSES	##### - OVERFLOW, FRAME, PARITY ERRORS
##### - EXCEPTION RESPONSES RECEIVED	##### - MESSAGE RETIRES

Baud Rate ##### Retries ### Timeout ### Data Bits 8 Parity None

Continued on next page

5.3 NDT Status Detail Displays, Continued

Status Display for Modbus Device

Note: The numbers in parentheses refer to the Dnn and FC07 enable codes you entered for the device NDT configuration.

Note: When the value displayed for a loopback test data counter is minus one (-1), that diagnostic code is not configured or supported for the device.

(M_DEVICE.ZS)

```
##### - MESSAGES TRANSMITTED      ##### - MESSAGE CRC ERRORS
##### - MESSAGES RECEIVED          ##### - PROTOCOL ERRORS
##### - NO RESPONSES               ##### - OVERFLOW, FRAME, PARITY ERRORS
##### - EXCEPTION RESPONSES RECEIVED ##### - MESSAGE RETIRES
```

DIAGNOSTIC LOOPBACK TEST DATA FROM THIS DEVICE:

```
##### - QUERY DATA (D00)          ##### - BUS MESSAGES (D11)
##### - BUS CRC ERRORS (D12)        ##### - BUS EXCEPTION ERRORS (D13)
##### - SLAVE MESSAGES (D14)        ##### - SLAVE NO RESPONSES (D15)
##### - SLAVE NAKS (D16)            ##### - SLAVE BUSY COUNT (D17)
##### - BUS CHAR. OVERRUNS (D18)    ##### - OVERRUN ERRORS (D19)
```

```
##### - DIAGNOSTIC REGISTER (D02)  DIAGNOSTIC REGISTER BY BIT:
BIT NO.  00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15
        ### ### ### ### ### ### ### ### ### ### ### ### ### ### ###
```

- READ EXCEPTION STATUS COIL (FC07), COIL BY VALUE:

```
COIL NO.   1   2   3   4   5   6   7   8
VALUE      ### ### ### ### ### ### ###
```

5.4 PITG and PIT – Define and Configure

TDFs for PITG and PIT templates

The TDFs for both the PITG and PIT templates are common across all CLM implementations.

Instructions and listings for the PITG and PIT Template Definition Files are detailed in the *CLM Platform Implementation Manual*. Refer to it for additional information.

The PIT effectively combines the device address including CLMI, port, and device. When you build PITEs for image points, you enter the image point address as PIT name, and coil or register address. The PIT table contains one or more points for a single device only.

Define the PITs

Each of the NDTs previously defined at the device level (M_DEVICE) must have at least one PIT assigned to it. This allows scheduling of data that is resident in that device. Multiple PITs may be assigned to a single device.

For more information on the PIT, refer to the *CLM Platform Implementation Manual*.

The following is an example of a typical PIT assigned to a Modbus device in a CLM/Modbus application.

```
ENTRY      = PIT
NAME       = PIT_DEV1
CYCLE      = 10S
CYC_ALM    = 5
SLO_DWN    = 6
NDT_NAME   = DEV1      (name of associated NDT entry)
END
```

STOP!

This completes the definition and configuration section of the Modbus application.

Although additional entries may be added at a later time, at this point you should review your NDT and PIT network entries.

It is important that this network be properly defined before you begin to configure PIT entries (PITEs). Once PITEs are built into the Network Memory structure, the basic structure of the NDTs cannot be changed without deleting all PITEs assigned to the associated PIT.

At least one PIT must be defined for each device before continuing on with Section 6. Refer to the *CLM Platform Implementation Manual*.

Section 6 – Modbus Image Point Interface Design

6.1 Overview

Introduction

The Point Interface Tables (PIT) and associated CLM structures define the mapping from the Modbus device data to the CLM image points. The standard CLM defines three structures (PIT Group, PIT, and PIT Entry). The use of the PIT Group is optional. The PIT defines an NDT-based table to which PIT Entries are added. The PIT Entries (PITEs) define the instructions for mapping device data into the CLM image point parameters. Each PIT and PITE is defined to the CLM through the CLM Configuration Builder.

Image point design and configuration process overview

Following below is an outline of the steps used in the design and configuration of an image point interface to a Modbus device.

You may need to review the following documents.

- *CLM Platform Implementation Manual*
- User's Manuals for your remote devices.

Note: Before designing and configuring image points, you must have completed Network Design and Configuration - Section 5.

Overview: Steps to Design and Configure Image Point Interface

Step	Activity
1	Identify the data from the Modbus device that are to be reflected in AM image points.
2	Define the AM image points.
3	Identify the TDFs for each type of image point. These TDFs will be used in building the PIT structures. • The TDFs for the PITG and PIT are listed in the <i>CLM Platform Implementation Manual</i>. • Supplied PITEs are identified in Table 3.3-1. The TDF files are a part of the Modbus application package and can be viewed on your system HM.
4	Identify the specific Modbus data in the network that maps to AM image point parameters.

Continued on next page

6.1 Overview, Continued

5	Identify the Modbus register number. The user enters the relative address of the register within the functional code range into the AD1 field within the PITE TDF (i.e. relative 3434 in Holding Registers is absolute address 33434) . Note: In the actual TDF for the image point, AD1 is usually renamed to a functional, descriptive field name. The register range (also defining the Functional Code) is entered into the AD2 field within the PITE TDF.
6	Step 7 and the creation of a BIF is optional. You can configure the network on-line using the interactive build feature of the CLM Configuration Builder. Add data to a BIF for each of the identified image points. • The BIF may be edited using the TPS file editor, or other text file editor. • The prototype BIFs included by Honeywell are identified in Table 3.3-1. • Actual BIF listings for each point type are included in Section 6.5. • Copy the appropriate BIF file and complete each definition within the BIF to create the BIF PITE entry. • Each PITE entry must be input to the CLM Configuration Builder to generate the PITE interface structure required for each imaged point.
7	Build the image points on the target CLM node. This is performed using the Data Entity Builder. • Reference the <i>Data Entity Builder Manual</i>.
8	Build the point interface structures on the target CLM node using the CLM Configuration Builder, which is installed on the CLM node as part of the CLM platform software. • Reference the <i>CLM Platform Implementation Manual</i>.

6.2 Modbus Data Types and Addresses

Introduction

Values are identified in the Modbus device by function or data type and address.

Register Addressing

Modbus Protocol Convention typically addresses registers as follows:

Cross Reference: Modbus Register vs. Function Codes

Modbus Device Register	Description
0 - 9999	Coils: Logical bits for read or write FC01 – Input FC05 – Single Output FC15 – Multiple Output (Array)
10000 - 19999	Input Status: Logical bits for read only FC02 – Input
30000 - 39999	Input Registers: Numeric values for read only FC04 – Input
40000 - 49999	Holding Registers: Numeric values for read or write FC03 – Input FC06 – Single Output FC16 – Multiple Output (Array)

Extended Modbus registers

Modern devices may support more than 10,000 coils, input status, input registers, and holding registers. The CLM/Modbus Application is therefore designed not to be bound by Modbus register addressing by convention.

CLM extended register limit

The range of Modbus register addresses in the CLM is 1 to 65535.

The address range code in each of the PITE templates restricts the address range to 1 – 10000 (Which identifies 0 – 9999). If a device is used for which a larger address is required (greater than our 10000), the user must modify the template and add a larger range code for the Modbus address. Refer to the *CLM Platform Implementation Manual*.

It needs to be pointed out that even though extended ranges are supported, due to the CLM maximum register address of 65535, a full 6-digit address range would not be possible.

Continued on next page

6.2 Modbus Data Types and Addresses, Continued

Modbus Data types

When building a PITE, you are prompted to enter the Modbus address and the Modbus data type.

Modbus Data Types for the CLM

Data Type	Description
1	Coils
2	Input Status
3	Holding Registers
4	Input Registers

The Modbus data type refers to the type of data being read; e.g., for coils the data type is 1, for holding registers the data type is 3.

Modbus Convention Addresses

In the CLM, register addressing is entered as a one-based relative address, as opposed to normal Modbus addressing convention, which is zero-based.

The following table contains information for those who are accustomed to entering Modbus addresses by convention.

Addressing: Modbus vs. CLM Convention

Modbus Device Address Range	Use Address	Use Datatype for Input and Output (PXX_AD2)
0 - 9999	1 - 10000	1
10000 – 19999	1 - 10000	2
30000 – 39999	1 - 10000	4
40000 - 49999	1 - 10000	3

NOTE that data types for holding and input registers are opposite of standard Modbus data types.

Continued on next page

6.2 Modbus Data Types and Addresses, Continued

Examples

If the Modbus address to be read is 10004 and it is mapped to an AM Flag point, using the M_FLAG.ZT as the PITE definition file, the following values apply:

PITE Definition File for M_FLAG.ZT

Field	Value	Comments
PV	5	Input status address (MB relative address 0004)
P01_AD2	2	Input status datatype (MB device register 10004)
P01_ICNV	5,2-2	Input conversion codes. Ref. 6.3.
P01_OCNV	..	There is no output to this datatype. “..” indicates no conversion.

If the Modbus address to be read is 40105 and it is mapped to an AM Regulatory point, using M_AI.ZT as the PITE definition file, the following values apply::

PITE Definition File for M_AI.ZT

Field	Value	Comments
PV	106	Holding register address (MB relative address 0105)
P01_AD2	3	Holding register data type (MB device register 40105)
P01_ICNV	2-3, 6, 0	Input conversion codes. Ref. 6.3. (16 bit unsigned integer)
P01_OCNV	2-3, 6, 0	Output conversion codes. Ref. 6.3.

6.2.1 Modbus Data Formats

Big Endian and Little Endian

Big Endian and Little Endian are byte order representations for a register or a set of registers.

In a Big Endian scheme, four bytes are represented as bytes 1, 2, 3, 4. Register N contains bytes 1 and 2 and register N + 1 contains bytes 3 and 4. The first byte is the *most* significant.

Byte 1	Byte 2	Byte 3	Byte 4
Register N		Register N+ 1	

In a Little Endian scheme, four bytes are represented as byte 4, 3, 2, 1. Therefore, register N contains bytes 4 and 3, and register N + 1 contains bytes 2 and 1. The first byte is the *least* significant.

Byte 4	Byte 3	Byte 2	Byte 1
Register N		Register N+ 1	

Big and Little Endian formats apply to single integers (two bytes) taking one register, to long integers (four bytes) taking two registers, and also to double precision floating point numbers (eight bytes) taking four registers.

Integer Data

Integer data requires one Modbus register (two bytes).

First Register

Hi Byte	Lo Byte
---------	---------

String Data

String data requires one Modbus register (two bytes) for each two characters. If the number of characters requested is odd, the correct number of characters will be read, but only an even number (one less) of characters will be written.

First Register

Last Register

1st Char	2nd Char	•	•	•	N - 1 Char	Last Char
----------	----------	---	---	---	------------	-----------

Continued on next page

6.2.1 Modbus Data Formats, Continued

Floating point data

IEEE floating point can be stored in five different ways, depending on the floating point parameter entered in the conversion code. They differ by byte significance and by length. A floating point number may take either 4 bytes or 8 bytes.

NOTE: The 'F' value in parentheses in the following examples indicates the CLM conversion value. Reference 6.3.

Big Endian floating point (F=1)

Big Endian Modbus Floating Point Data Format

Sign/Exp.	Mantissa 1	Mantissa 2	Mantissa 3
Byte 1	Byte 2	Byte 3	Byte 4
Register N		Register N+ 1	

Little Endian floating point (F=2)

Little Endian Modbus Floating Point Data Format

Mantissa 3	Mantissa 2	Mantissa 1	Sign/Exp.
Byte 1	Byte 2	Byte 3	Byte 4
Register N		Register N+ 1	

This format is used by some MVIP devices such as the Triconex CPC and the Honeywell 620 LCS PLC.

Continued on next page

6.2.1 Modbus Data Formats, Continued

Modbus word swapped format (F=3)

The Modicon floating point format is also equivalent to an IEEE format for a floating point representation. The four bytes for a Modicon floating point number are represented as bytes 3,4,1,2, with register N having bytes 3 and 4, and register N + 1 containing bytes 1 and 2. The first byte is the most significant, with the high and low words swapped.

Modicon Floating Point Data Format

Mantissa 2	Mantissa 3	Sign/Exp.	Mantissa 1
Byte 3	Byte 4	Byte 1	Byte 2
Register N		Register N+ 1	

This format is used by the Modicon 984 PLC.

Big Endian format, double word (F=4)

Big Endian Double Word Floating Point Data Format

Sign/ Exp.	Exp.	M 1	M 2	M 3	M 4	M 5	M 6
Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7	Byte 8
Register N		Register N+1		Register N+2		Register N+3	

Little Endian format, double word (F=5)

Little Endian Double Word Floating Point Data Format

M 6	M 5	M 4	M 3	M 2	M 1	Exp.	Sign/ Exp.
Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7	Byte 8
Register N		Register N+1		Register N+2		Register N+3	

This format is used by some MVIP devices such as the Triconex CPC and the Honeywell 620 LCS PLC.

6.3 LCN/Modbus Conversion Codes

Introduction

Values input from and output to Modbus devices must be converted to or from LCN formats. Specific codes define the required conversions.

CLM platform supported conversions

Conversion codes are specified in two parts that are separated by a dash. The first part is the LCN data format - Table 6.3-1. The Modbus data format - Table 6.3-2 is the second part of the total PITE parameter conversion definition. Refer to the definition of the Pnn_ICNV and Pnn_OCNV in the PITE template definition section of the *CLM Platform Implementation Manual* for details.

FORMAT: <LCN CODE> - <MODBUS CODE> , <CODE OPTIONS>

These conversion codes are included in the PITE template definition. Each PITE parameter can have input and/or output conversions.

LCN Conversions

The LCN conversion code is the first part of the two part PITE parameter conversion definition and determines how the input data will be converted to the LCN parameter.

Table 6.3-1 LCN/CLM Conversion Codes

LCN Conversion Code	Data Type
Platform Conversions (for Reference)	
2	Analog value
3	Logical value
4,N	String value (N=# of characters). For Modbus, this requires N/2, rounded up, consecutive registers.
5,max	LCN Enumeration (max is the number of states)
7,N	Analog Array (N is the number of elements)
8,N	Logical Array (N is the number of elements)

Continued on next page

6.3 LCN/Modbus Conversion Codes, Continued

Modbus conversions The Modbus conversion code is the second part of the total PITE parameter conversion definition, and determines how the data in the message is to be handled.

Table 6.3-2 CLM Supported Modbus Conversions

Modbus Device Conversion Code	Data Type										
1	Read or Write Coil Status (must be consecutive coils for array data)										
1,13	Read or Write Coil with logical inversion (must be consecutive coils for array data). Inversion is identified as when the Modbus Device bit is OFF the CLM result will be ON, and when the bit is ON it will be represented as OFF.										
2	Read Input Status (must be consecutive registers for array or enumeration data)										
2,13	Read Input Status with logical inversion (See 1,13)										
3	Read or Write to consecutive holding registers for string or enumeration data.										
3,13,bit 4,13,bit	Read with logical inversion of a single bit from one or more holding or input registers. This conversion code allows you to extract a single bit within a register. Bit 0 is defined as the least significant bit. (Inversion identified above). NOTE: Single bits may be extracted from holding or input registers, but no writes are allowed to single bits in either register type.										
3,F 4,F	Read or Write 1 to N consecutive holding registers or input registers for floating point data. Parameter F is defined below. NOTE: N items at the LCN translates to N times the number of Modbus registers per value.										
Floating point parameter F <table border="1"> <tr> <td>1</td><td>IEEE Floating Point Representation, Big Endian (Motorola) Format, two registers per value.</td></tr> <tr> <td>2</td><td>IEEE Floating Point Representation, Little Endian (Intel) Format, two registers per value.</td></tr> <tr> <td>3</td><td>Modicon Floating Point , two registers per value.</td></tr> <tr> <td>4</td><td>IEEE Double Precision Real, Big Endian (Motorola) Format, four registers per double value.</td></tr> <tr> <td>5</td><td>IEEE Double Precision Real, Little Endian (Intel) Format, four registers per double value.</td></tr> </table>		1	IEEE Floating Point Representation, Big Endian (Motorola) Format, two registers per value.	2	IEEE Floating Point Representation, Little Endian (Intel) Format, two registers per value.	3	Modicon Floating Point , two registers per value.	4	IEEE Double Precision Real, Big Endian (Motorola) Format, four registers per double value.	5	IEEE Double Precision Real, Little Endian (Intel) Format, four registers per double value.
1	IEEE Floating Point Representation, Big Endian (Motorola) Format, two registers per value.										
2	IEEE Floating Point Representation, Little Endian (Intel) Format, two registers per value.										
3	Modicon Floating Point , two registers per value.										
4	IEEE Double Precision Real, Big Endian (Motorola) Format, four registers per double value.										
5	IEEE Double Precision Real, Little Endian (Intel) Format, four registers per double value.										

Continued on next page

6.3 LCN/Modbus Conversion Codes, Continued

Table 6.3-2 CLM Supported Modbus Conversions, continued

Modbus Device Conversion Code	Data Type																						
3,F,SF 4,F,SF	Read or Write 1 to N holding registers for integer Data. Parameters F and SF are defined below. NOTE: N items at the LCN translates to N Modbus registers per item.																						
Parameter F must be 6 or 7 for integer data:																							
<table><tr><td>6</td><td>Unsigned Integer (single register)</td></tr><tr><td>7</td><td>Signed Integer (single register)</td></tr></table>		6	Unsigned Integer (single register)	7	Signed Integer (single register)																		
6	Unsigned Integer (single register)																						
7	Signed Integer (single register)																						
SF, scaling factors for Modbus numeric data:																							
<table><tr><td>0</td><td>No scaling</td></tr><tr><td>1</td><td>Divide by 100,000</td></tr><tr><td>2</td><td>Divide by 10,000</td></tr><tr><td>3</td><td>Divide by 1000</td></tr><tr><td>4</td><td>Divide by 100</td></tr><tr><td>5</td><td>Divide by 10</td></tr><tr><td>6</td><td>Multiply by 10</td></tr><tr><td>7</td><td>Multiply by 100</td></tr><tr><td>8</td><td>Multiply by 1000</td></tr><tr><td>9</td><td>Multiply by 10,000</td></tr><tr><td>10</td><td>Multiply by 100,000</td></tr></table>		0	No scaling	1	Divide by 100,000	2	Divide by 10,000	3	Divide by 1000	4	Divide by 100	5	Divide by 10	6	Multiply by 10	7	Multiply by 100	8	Multiply by 1000	9	Multiply by 10,000	10	Multiply by 100,000
0	No scaling																						
1	Divide by 100,000																						
2	Divide by 10,000																						
3	Divide by 1000																						
4	Divide by 100																						
5	Divide by 10																						
6	Multiply by 10																						
7	Multiply by 100																						
8	Multiply by 1000																						
9	Multiply by 10,000																						
10	Multiply by 100,000																						

Beware!

The conversion codes following allow bit (and range of bits) manipulation. This includes both read and write capability in most cases. Although the Modbus protocol allows the ability to write a single bit or range of bits to a Holding register, this is not considered to be a safe write operation. Because this write is still a full register operation, there is a real danger of unaffected bits changing, resulting in an unexpected value. Honeywell does not recommend the use of this type of write operation.

Continued on next page

6.3 LCN/Modbus Conversion Codes, Continued

Table 6.3-2 CLM Supported Modbus Conversions, continued

Conversion Code	
Modbus	Data Type
3,8,start,stop,SF 4,8,start,stop,SF	Read or Write subfields of one or more holding registers or input registers for integer data. Writes are to holding registers only. This conversion code allows you to extract a value from a range of bits within the register. Start and Stop are bit numbers (0-15) with Bit 0 being the least significant bit. The resulting value can also be scaled with the SF defined above. NOTE: N items at the LCN translates to N Modbus registers per item.
3,9,num,SF 4,9,num,SF	Read or Write one or more holding registers for integer data. Num is the number of bits to be extracted starting with the least significant bit of the register. For arrays, num bits of a register are used for an element of an array. For the next element of an array, the next num bits of the same register are used. If there are no more bits left in this register, the next sequential register is used. Parameter SF is defined above. Num can be 1, 2, 4, 8, or 16. NOTE: N items at the LCN translates to N Modbus registers per item.
3,10,num,SF 4,10,num,SF	Read or Write one or more holding registers for integer data. Num is the number of bits to be extracted starting with the most significant bit of the register. For arrays, num bits of a register are used for an element of an array. For the next element of an array, the next num bits of the same register are used. If there are no more bits left in this register, the next sequential register is used. Parameter SF is defined above. Num can be 1, 2, 4, 8, or 16. NOTE: N items at the LCN translates to N Modbus registers per item.

Continued on next page

6.3 LCN/Modbus Conversion Codes, Continued

Table 6.3-2 CLM Supported Modbus Conversions, continued

Conversion Code	
Modbus	Data Type
3,11,SF 4,11,SF (See Note A)	For use with the big endian format for long integers. Read or Write two or more holding registers or input registers for <u>signed</u> long (4-byte, 2 register) integer data. Writes are to holding registers only. Parameter SF is defined above. Range: -21474836 to 21474836 NOTE: N items at the LCN translates to 2*N Modbus registers per item.
3,12,SF 4,12,SF (See Note A)	For use with the little endian format for long integers. Read or Write two or more holding registers or input registers for <u>signed</u> long (4-byte, 2 register) integer data. Writes are to holding registers only. Parameter SF is defined above. Range: -21474836 to 21474836 NOTE: N items at the LCN translates to 2*N Modbus registers per item.

Note A: Although the Modbus application supports the range shown for long integers, the view from the user station on the LCN will support only 6 digits of accuracy plus a sign bit. However, all values within the specified range will be sent to the Modbus device with full accuracy when entered by the user.

Continued on next page

6.3 LCN/Modbus Conversion Codes, Continued

Table 6.3-3 Conversion Code Examples

LCN Type	Modbus Type	ICNV or OCNV	Description
Analog Value	Input Register	2-4,6,7	LCN analog (2) - Modbus Input Register (4), Single unsigned integer (6), scaled @ * 100 (7)
Analog Value	Input Register	2-4,3	LCN analog (2) - Modbus Input Register (4), IEEE Modicon floating point value (3)
Analog Value	Input Register	2-4,1	LCN analog (2) - Modbus Input Register (4), IEEE floating point value big endian type (1)
Analog Value	Holding Register	2-3,6,0	LCN analog (2) - Modbus Holding Register (3), Single unsigned integer (6), no scaling (0)
Analog Value	Holding Register	2-3,8,3,6	LCN analog (2) - Modbus Holding Register (3), Subfields (8), start bit (3), stop bit (6) (First bit is least significant, bit zero is least significant in conversion code)
Analog Value	Holding Register	2-3,11,0	LCN analog (2) - Modbus Holding Register (3), Signed Double Integer in big endian type (11), no scaling (0)
Analog Value	Input Register	2-4,12,3	LCN analog (2) - Modbus Input Register (4), Signed Double Integer in little endian type (12), scaled to divide by 1000 (3).
Analog Array	Holding Register	7,500-3,3	LCN analog array (7), (500) elements - Modbus Holding Register (3), Modicon floating point. (3)
Analog Array	Holding Register	7,20-3,10,4,6	LCN analog array (7), (20) elements - Modbus Holding Register (3), multiple registers (10), number of bits (4), scaled to multiply by 10 (6)
Analog Array	Holding Register	7,17-3,2	LCN analog array (7), (17) elements - Modbus Holding Register (3), IEEE floating point value type little endian (2)
Analog Array	Holding Register	7,20-3,6,0	LCN analog array (7), (20) elements - Modbus Holding Register (3), Single unsigned integer (6), no scaling (0)

Continued on next page

6.3 LCN/Modbus Conversion Codes, Continued

Table 6.3-3 Conversion Code Examples, continued

LCN Type	Modbus Type	ICNV or OCNV	Description
Logical	Coil	3-1	LCN logical (3) - Modbus coil (1)
Logical	Input Status	3-2	LCN logical (3) - Modbus Input Status (2)
Logical	Coil	3-1,13	LCN logical (3) - Modbus coil (1), Logically inverted (13) (Can read or write coil)
Logical	Input Register	3-4,8,0,0	LCN logical (3) – Modbus Input Register (4), Subfields (8), Start bit (0), Stop bit (0) --i.e. Extract a single packed bit--
Logical Value	Holding or Input Register	2-3,13,4 2-4,13,4	LCN analog (2) – Modbus Holding Register (3) or Modbus Input Register (4) – Logical inversion (13) – Bit identification (4) – Bit 0 being the least significant bit – Read of bit 4 with logical inversion. – bit 4 is the 5 th bit starting from the least significant bit (0, 1, 2, 3, 4).
Enumeration (Flag)	Coil	5,2-1	LCN enumeration (5), (2) states - Modbus Coil (1)
Enumeration (Flag)	Input Register	5,2-4,8,10,10	LCN enumeration (5), (2) states - Modbus Input Register (4), Subfields (8), Start bit (10), Stop bit (10) --i.e. Extract the 11 th packed bit--
Enumeration (Flag)	Input Status	5,2-2	LCN enumeration (5), (2) states - Modbus Input Status (2)
Logical Array	Coil	8,100-1	LCN Logical array (8), (100) elements – Modbus Coil (1) for read or write
String	Holding Register	4,10-3	LCN string (4), (10) characters – Modbus Holding Register (3) (String is in five holding registers - 10/2)

Note: Scaling on input of a value will require reverse scaling prior to output.

6.4 PITE Level Options

Introduction

PITE level options provide control over the input and output process.
Reference the *CLM Implementation Manual*.

Input Parameters - Exceptions

Modbus exceptions: Input PITE Parameter Options

Input PITE Parameter Options (IFNC)	
ECP	Unsolicited or RBE input processing. NOT SUPPORTED for Modbus.

Output Parameters - Exceptions

Modbus exceptions: Output PITE Parameter Options

Output PITE Parameter Options (OFNC)	
CBO	Check before operate. NOT SUPPORTED for Modbus.

6.5 Modbus Image Point Types

Image points

- 6.5.1 Analog Input
 - 6.5.2 Analog Output
 - 6.5.3 Accumulator
 - 6.5.4 Numeric Point
 - 6.5.5 Digital Input (3- or 4-state)
 - 6.5.6 Digital Composite (3-, 4-, or 5-state)
 - 6.5.7 Flag Point (Single digital input/output)
 - 6.5.8 Message Point (ASCII String)
 - 6.5.9 Array Input / Output
-

Point configuration examples

In each of the examples of point configuration in the following sections, all items (parameters) available on the builder form are not necessarily identified. Only the minimum parameters required to properly setup the required structure are shown. Other parameters may be entered by the user for their specific implementation.

Image Point Section Components

Part	Description
1	Theory of operation
2	The point configuration data visible to the Data Entity Builder
3	Custom Data Segment parameters on the Point Detail Display
4	The builder input file format
5	CL blocks, if any, to be attached (Refer to Table 3.3-1 for a list of the unlink/link EC files included.)

PITE template design

The supplied Modbus PITEs are designed as follows:

- Point scheduling is defined in the PIT. The AM image point is processed after input by always setting the point option PPS=YES.
 - A conversion code must be entered for each Modbus address. With multiple possibilities for both discrete and analog data types, it is not reasonable to predetermine the conversion strings.
-

Continued on next page

6.5 Modbus Image Point Types, Continued

Using the link EC file

After you build your AM image point, you may need to attach CL blocks as specified in the following subsections. To use the EC file provided with the Modbus package, do the following:

1	Set path for CL SOURCE/OBJ to UCLM or the directory where you installed the CL.
2	Set the CL CUSTOM GDF to CDSG or your choice.
3	Execute the EC file with the image point name as a parameter. For example: EC M_TZ.EC IMAGE_PT

6.5.1 Analog Input Point (AM Regulatory Point Type)

Introduction

The Analog Input is an AM point that images a single numeric value in a register of the Modbus device. This section describes how to create and configure this point type. Note that all parameters below are not necessarily shown unless they are required or may be utilized by the design engineer.

Theory of operation

The Analog Input point operates by mapping the register in the Modbus device directly to the INPUT custom parameter in the attached M_AI Custom Data Segment (CDS) package. The image point is configured to have no scheduling period, but is PPS'd by the CLM Platform software when new data is available from the Modbus interface. The point then executes, causing the Data Acquisition (DATAACQ) algorithm to move the raw data to the PV of the point. Use of trip limits and alarming is available since the image point is AM-based. If communications errors occur, the INPUT parameter may be set to BAD by the CLM Platform software, thus propagating a BAD value to the PV.

Point configuration – Build the image point

Analog Input Point Configuration

Using the LCN Data Entity Builder, an AM image point must be built, using the table following as a template for data entry. Entries left blank are either optional or user defined.

Parameter	Description	Entry	Notes
NAME	POINT NAME		
UNIT	UNIT ID		
PTDESC	POINT DESCRIPTOR		
EUDESC	E.U. DESCRIPTOR		
KEYWORD	POINT KEYWORD		
PRIMMOD	PRIMARY MODULE POINT ID	----	
PTDISCL	POINT DISCLOSURE	FULL	
PVALGID	PV ALGORITHM	DATAACQ	
CTLALGID	CONTROL ALGORITHM	NULL	
OVERVAL	OVERVIEW DISPLAY VALUE		
SUPPIO	I/O DISPLAY SUPPRESSION	OUTSUPPR	
PERIOD	POINT EXECUTION PERIOD	NOPERIOD	CLM S/W will PPS

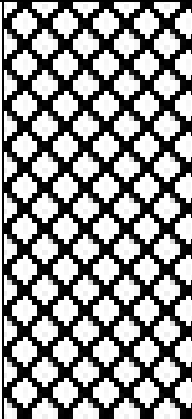
Continued on next page

6.5.1 Analog Input Point (AM Regulatory Point Type), Continued

Point configuration Analog Input Point Configuration, continued

Parameter	Description	Entry	Notes
NOPINPTS	NO. OF INPUT CONNECTIONS	1	From M_AI CDS package
PISRC1	SOURCE POINT ID.PARAMETER 1	*.INPUT	
PIDSTN1	DESTINATION PARAMETER	P1	
PIACTSTS1	DEST CONN ACTIVITY STATUS 1	ACTIVE	
CLSLOTS	NUMBER OF CL BLOCKS	0	Req'd on all CLM points
NOPKG	NO. OF CUSTOM DATA PACKAGES	2	
NUMSWTCH	NUMBER OF SWITCHES	0	
NOGINPTS	NUMBER OF GENERAL INPUTS	0	
NOGOPTS	NUMBER OF GENERAL OUTPUTS	0	
PKGNAME(1)	SOURCE FILE 1 NAME	M_AI	
PKGNAME(2)	SOURCE FILE 2 NAME	CLM_DMD3	

CDS parameters Analog Input Point CDS Parameters

Parameter	Description	Entry	Notes
M_AI	CDS Parameters on PV Algorithm Page		
INPUT	POINT VALUE		Updated from MB data
CLM_DMD3	CDS Parameters on Custom Page		
CLM_PPS	STORE TO PROCESS POINT PITE		Special Applications
CLM_OUT	STORE TO PROCESS OUTPUT PITE		Special Applications
DEV_STA	DEVICE IN_SERV (ON/OFF)		Updated by CLM S/W
PT_FAIL	DEVICE COMPONENT FAILURE		Not used for Modbus
HDW_REF	HARDWARE REFERENCE		Not used for Modbus
CLM_PORT	NAME OF PORT NDT		Updated by CLM Builder
CLM_DEV	NAME OF DEVICE NDT		Updated by CLM Builder
CLM_PIT	NAME OF PIT TABLE		Updated by CLM Builder

Continued on next page

6.5.1 Analog Input Point (AM Regulatory Point Type), Continued

Builder input file – Map the image point

Analog Input Point Builder Fields

Using the CLM Builder, the Modbus data for the Analog Input type point must be mapped from the LCN parameters to the Modbus device registers.

Prompt	Entry	Description
ENTRY	M_AI	Template Name (.ZT)
NAME		LCN Analog Input point name
PIT_NAME		PIT name
INPUT		Register to read the analog value from
P01_AD2		Enter 3 if Holding register, 4 if Input register
P01_ICNV		Data conversion code. See Section 6.3.
END		End of current entry

6.5.2 Analog Output Point (AM Regulatory Point Type)

Introduction

The Analog Output is an AM point that images a Modbus register as an analog value and is used to output a value from the image point to that register. This section describes how to create and configure this point type. Note that all parameters below are not necessarily shown unless they are required or may be utilized by the design engineer.

Theory of Operation

The Analog Output point maps the SP of a Regulatory AM point (Null PV and CL Control Algos) to the desired Modbus register. Feedback is used to validate the completed store by mapping a CDS parameter OUTPUT to the register (normally the same as the SP item).

The point must be in AUTO mode in order to output a SP value. The SP engineering unit range (SPEULO, SPEUHI) must be set up, as should the CV range since it is defined as Engineering Units type.

With the point in AUTO, the operator changes the SP. This value is sent by “exception processing” to the Modbus register. After a configurable time (RB_TIME in the Template file), a readback command is initiated automatically by the CLM software and the register is read back in to the OUTPUT parameter and the point PPS'd.

A General Input (GI) moves the OUTPUT data to the CV and causes the OP to update with the proper range percentage value.

Note: Outputs are not sent out in the MAN mode.

Earlier AO Versions

Important! The Analog Output point has been re-engineered to no longer require an attached CL block as it has in the past. It does still use the same attached CDS packages. There is also a new template file that is used for mapping the point data called M_AO2. The files used for the earlier versions of the Analog Output point are still resident on the release media if needed.
--

Continued on next page

6.5.2 Analog Output Point (AM Regulatory Point Type),

Continued

Point configuration – Build the image point

Analog Output Point Configuration

Using the LCN Data Entity Builder, an AM image point must be built, using the table following as a template for data entry. Entries left blank are either optional or user defined.

Parameter	Description	Entry	Notes
NAME	POINT NAME		
UNIT	UNIT ID		
PTDESC	POINT DESCRIPTOR		
EUDESC	E.U. DESCRIPTOR		
KEYWORD	POINT KEYWORD		
PRIMMOD	PRIMARY MODULE POINT ID	----	
PTDISCL	POINT DISCLOSURE	FULL	
PVALGID	PV ALGORITHM	NULL	
CTLALGID	CONTROL ALGORITHM	CL	
OVERVAL	OVERVIEW DISPLAY VALUE		
SUPPIO	I/O DISPLAY SUPPRESSION	INPSUPPR	
PERIOD	POINT EXECUTION PERIOD	NOPERIOD	
NMODE	POINT'S NORMAL MODE	AUTO	Must be AUTO to output a value from the SP
NMODATTR	NORMAL MODE ATTRIBUTE	OPERATOR	
MODEPERM	MODE PERMISSIVE	PERMIT	
EXTSWOPT	EXT MODE SWITCHING OPTION	NONE	
SPFORMAT	SP DECIMAL POINT FORMAT		
SPEUHI	SP EU HIGH RANGE in EUs		
SPEULO	SP EU LOW RANGE in EUs		
SPHILM	SP HIGH LIMIT		Set equal to SPEUHI
SPLOLM	SP LOW LIMIT		Set equal to SPEULO
SP	SET POINT		Value sent to register
SPOPT	SET POINT OPTION	NONE	

Continued on next page

6.5.2 Analog Output Point (AM Regulatory Point Type),

Continued

Point configuration

Analog Output Point Configuration, continued

Parameter	Description	Entry	Notes
CVTYPE	TYPE OF CV	ENGRUNIT	Set equal to SPEUHI Set equal to SPEULO
INITTYPE	INITIALIZATION TYPE	EXT	
ARWDI	ANTI-RESET WINDUP DIRECTION	DIRECT	
NOCOPTS	NUMBER OF OUTPUT CONNECTIONS	0	
CVEUHI	CV ENGINEERING UNITS HIGH		
CVEULO	CV ENGINEERING UNITS LOW		
OP	CONTROL OUTPUT		
OPHILM	OP HIGH LIMIT VALUE IN %		
OPLOLM	OP LOW LIMIT VALUE IN %		
OPMCHLM	OP MINIMUM CHANGE IN %		
CLSLOTS	NUMBER OF CL BLOCKS	0	
NOPKG	No. OF CUSTOM DATA PACKAGES	2	
NUMSWTCH	NUMBER OF SWITCHES	0	
NOGINPTS	NUMBER OF GENERAL INPUTS	1	
NOGOPTS	NUMBER OF GENERAL OUTPUTS	0	
PKGNAME(1)	SOURCE FILE 1 NAME	M_AO	
PKGNAME(2)	SOURCE FILE 2 NAME	CLM_DMD3	
GISRC(1)	GENERAL INP SOURCE PAR	*.OUTPUT	Feedback parameter
GIDSTN(1)	GENERAL INP DESTINATION PAR	CV	Push to OP for % range
GIACTSTS(1)	GENERAL INP ACTIVATION STS	ACTIVE	

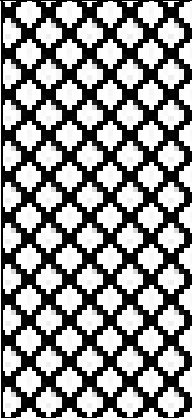
Continued on next page

6.5.2 Analog Output Point (AM Regulatory Point Type),

Continued

Point configuration

Analog Output Point Configuration, continued

Parameter	Description	Entry	Notes
CDS Parameters (M_AO) on the Control Algorithm Page			
OUTPUT	CURRENT OUTPUT VALUE		MB register feedback from SP data sent out
CDS Parameters (CLM_DMD3) on Custom Page			
CLM_PPS	STORE TO PROCESS POINT PITE		Special Applications
CLM_OUT	STORE TO PROCESS OUTPUT PITE		Special Applications
DEV_STA	DEVICE IN_SERV (ON/OFF)		Updated by CLM S/W
PT_FAIL	DEVICE COMPONENT FAILURE		Not used for Modbus
HDW_REF	HARDWARE REFERENCE		Not used for Modbus
CLM_PORT	NAME OF PORT NDT		Updated by CLM Builder
CLM_DEV	NAME OF DEVICE NDT		Updated by CLM Builder
CLM_PIT	NAME OF PIT TABLE		Updated by CLM Builder

Builder input file – Map the image point

Analog Output Point Builder Fields

Using the CLM Builder, the specific parameters of the Analog Output type point must be mapped to the required Modbus registers.

Prompt	Entry	Description
ENTRY	M_AO2	Template Name (.ZT)
NAME		LCN Analog Output point name
PIT_NAME		NDT PIT Table Name
SP_ADDR		Register to send value to
SP_CONV		Output Conversion Code
FB_ADDR		Feedback Register Address
REG_TYPE		Feedback Register Type (Input - 3 or Holding – 4)
FB_CONV		Input Conversion Code
END		End of current entry

6.5.3 Accumulator Point (Rate and Accumulation)

Introduction

The accumulator type point is used to image an accumulation process in the Modbus device. There are two scenarios in the device that are supported by the Modbus accumulation point type.

- Mapped register contains the Accumulation Rate
- Mapped register contains the actual Accumulation Counter

These two variations of the accumulator point are discussed below.

2 Image Point Types

For the Modbus application, the two scenarios above are emulated in two different types of image points.

- Accumulator Point – Generates an accumulated counter value based on the accumulation rate mapped from the device register.
 - Totalizer Point – Generates an accumulation rate based on the accumulation counter mapped from the device register.
-

Theory of Operation: Modbus register = Rate value (Accumulator)

In certain cases, the Modbus device keeps the accumulation rate in the register and the accumulation value is to be calculated in the image point. The accumulation rate is defined as a number of instances per specified time frame (i.e. pulses/sec, feet/min, etc.).

An AM Regulatory point is built using the Totalizer PV algorithm. CDS package M_TZ2 is added to house the mapped data from the Modbus register containing the rate value (AV).

The rate time base in the Modbus device must match the time base configured on the image point. The time base is available in seconds, minutes and hours only.

The period of the image point is configurable by the user, but must be less than or equal to the time base rate in the device.

The accumulator point must be RESET and STARTED by the user to begin the accumulation processing. The value read in from the Modbus device register is considered to be the rate value and the image point PV will accumulate accordingly.

Any changes in the rate value will cause the image point to be PPS'd and the new rate accumulation is immediately applied.

Continued on next page

6.5.3 Accumulator Point (Rate and Accumulation), Continued

**Theory of Operation:
Modbus register =
Accumulated value**

In other cases, the Modbus device keeps the accumulation total in the register as a rolling register value and the accumulation rate is to be calculated in the image point.

An AM Regulatory point is built using the Data Acquisition PV and CL Control algorithms. CDS package M_TZ2 is added to house the mapped data from the Modbus register containing the accumulation value (AV).

A CL block is attached that runs when the AV value is updated by the CLM. This CL block checks the time since the last update, compares the current accumulation value (AV) against the previous value (LAST_AV) and uses this to calculate the accumulation rate. This rate (PVRAW) is used as the P1 source to the PV algorithm and pushed to the PV.

**Theory of Operation:
Direct Emulation**

In certain cases, the complete accumulation function may be done in the Modbus device and the results (either the accumulation value or rate) placed into a register. These values may need to be displayed at the image point level. In this instance, a standard Modbus Analog Input type point (See Section 6.5.1) should be used, since there is not accumulation function actually being done in the image point.

**Determine your
Accumulation point
type**

At this point, you need to determine the type of accumulation process that is being utilized in the Modbus device. Based on this information, an Accumulator point should be built and mapped to emulate this process. Use the following table to determine which section should be used to build your Accumulator point.

If the Modbus Register contains	AND the Image Point needs to display	Continue with Section
Accumulation Rate	Accumulation Counter	6.5.3.1
Accumulation Rate	Accumulation Rate	6.5.1 (Analog Input)
Accumulation Counter	Accumulation Rate	6.5.3.2
Accumulation Counter	Accumulation Counter	6.5.1 (Analog Input)

6.5.3.1 Accumulator Point (Register = Rate Value)

Introduction

This section describes the steps and information required to emulate an accumulation process in the device that generates a rate in the mapped register.

The image point will use the rate from the mapped register to generate an accumulation counter value on the image point.

See previous Section 6.5.3 for specific details.

Point configuration - Build the image point

Accumulator Point Configuration (Register = rate value)

Using the LCN Data Entity Builder, an AM image point must be built, using the table following as a template for data entry. Entries left blank are either optional or user defined.

Parameter	Description	Entry	Notes
NAME	POINT NAME		
UNIT	UNIT ID		
PTDESC	POINT DESCRIPTOR		
EUDESC	E.U. DESCRIPTOR		
KEYWORD	POINT KEYWORD		
PRIMMOD	PRIMARY MODULE POINT ID	----	
PTDISCL	POINT DISCLOSURE	FULL	
PVALGID	PV ALGORITHM	TOTALIZR	
CTLALGID	CONTROL ALGORITHM	NULL	
OVERVAL	OVERVIEW DISPLAY VALUE		
SUPPIO	I/O DISPLAY SUPPRESSION	OUTSUPPR	
PERIOD	POINT EXECUTION PERIOD		Must be <= a single unit of the selected time base
PVEUHI	PV EU HIGH RANGE		Maximum accumulation (or rollover) value
PVEULO	PV EU LOW RANGE		Value >= 0

Continued on next page

6.5.3.1 Accumulator Point (Register = Rate Value), Continued

Point configuration Accumulator Point Configuration (Register = rate value), continued

Parameter	Description	Entry	Notes
PVEQN	PV ALGORITHM EQUATION	SECONDS MINUTES HOURS	Refer to <i>the AM Control Functions</i> LCN manual for the PV Totalizer algorithm
TIMEBASE	ACCUMULATION TIME BASE		MUST match with the time base value in the remote device
C	UNITS CONVERSION VALUE		Default = 1
RESETVAL	RESET VALUE FOR AV		Value >= 0 Placed in PV when RESET command issued
CUTOFFLM	CUTOFF LIMIT VALUE		Input below this value treated as zero
AVTV (SP)	ACCUMULATION TARGET VALUE		Target value for accumulation
AVDEV1TP	ACCUMULATION DEVIATION TRIP1		First SP-PV deviation alarm trip value ("Coarse" adjustment)
AVDEV2TP	ACCUMULATION DEVIATION TRIP2		Second SP-PV deviation alarm trip value ("Fine" adjustment)
NOPINPTS	NO. OF INPUT CONNECTIONS	1	From attached CDS
PISRC1	SOURCE POINT ID.PARAMETER 1	*.AV	
PIDSTN1	DESTINATION PARAMETER	P1	
PIACTSTS1	DEST CONN ACTIVITY STATUS 1	ACTIVE	
PVALDB	PV ALARM DEADBAND	ONE	As Required As Required
PVHITP	PV HIGH TRIP LIMIT	ENABLE	
PVLOTP	PV LOW TRIP LIMIT		
ALENBST	ALARM ENABLE STATE		

Continued on next page

6.5.3 Accumulator Point (Register = Rate Value), Continued

Point configuration Accumulator Point Configuration (Register = rate value), continued

Parameter	Description	Entry	Notes
CLSLOTS	NUMBER OF CL BLOCKS	0	
NOPKG	NUMBER OF CUSTOM DATA PACKAGES	2	
NUMSWTCH	NUMBER OF SWITCHES	0	
NOGINPTS	NUMBER OF GENERAL INPUTS	0	
NOGOPTS	NUMBER OF GENERAL OUTPUTS	0	
PKGNAME(1)	SOURCE FILE 1 NAME	M_TZ	
PKGNAME(2)	SOURCE FILE 2 NAME	CLM_DMD3	
CDS Parameters (M_TZ) on PV Algorithm Page			
AV	REMOTE ACCUMULATION RATE		Accumulation rate value from Modbus register
Note: The AV parameter above is the only parameter within the M_TZ2 CDS package that is used for this implementation of the Accumulator point. All other parameters in the CDS package should be left at their default values.			
CDS Parameters (CLM_DMD3) on Custom Page			
CLM_PPS	STORE TO PROCESS POINT PITE		Special Applications
CLM_OUT	STORE TO PROCESS OUTPUT PITE		Special Applications
DEV_STA	DEVICE IN_SERV (ON/OFF)		Updated by CLM S/W
PT_FAIL	DEVICE COMPONENT FAILURE		Not used for Modbus
HDW_REF	HARDWARE REFERENCE		Not used for Modbus
CLM_PORT	NAME OF PORT NDT		Updated by CLM Builder
CLM_DEV	NAME OF DEVICE NDT		Updated by CLM Builder
CLM_PIT	NAME OF PIT TABLE		Updated by CLM Builder

Continued on next page

6.5.3.1 Accumulator Point (Register = Rate Value), Continued

**Builder input file –
Map the image point****Accumulator Point Builder Fields**

Using the CLM Builder, the specific parameters of the Accumulator type point must be mapped to the required Modbus registers.

Prompt	Entry	Description
ENTRY	M_ACCUM	Template Name (.ZT)
NAME		LCN Point Name
PIT_NAME		NDT PIT Table Name
AV		Rate value register address
P01_AD2		Modbus data type (3 or 4)
P01_ICNV		Input conversion code
END		End of current entry

CL Block – Not Used

The CL block (M_TZ2CL.AO) provided with the Modbus package is **not** used with this implementation of the Accumulator point.

6.5.3.2 Totalizer Point (Register = Accumulated Value)

Introduction

This section describes the steps and information required to emulate an accumulation process in the device that generates an accumulated counter value in the mapped register.

The image point will use the counter value from the mapped register to generate an accumulation rate on the image point.

See previous Section 6.5.3 for specific details.

Point configuration – Build the image point

Totalizer Point Configuration (Register = accumulated value)

Using the LCN Data Entity Builder, an AM image point must be built, using the table following as a template for data entry. Entries left blank are either optional or user defined.

Parameter	Description	Entry	Notes
NAME	POINT NAME		
UNIT	UNIT ID		
PTDESC	POINT DESCRIPTOR		
EUDESC	E.U. DESCRIPTOR		
KEYWORD	POINT KEYWORD		
PRIMMOD	PRIMARY MODULE POINT ID	----	
PTDISCL	POINT DISCLOSURE	FULL	
PVALGID	PV ALGORITHM	DATAACQ	
CTLALGID	CONTROL ALGORITHM	CL	
OVERVAL	OVERVIEW DISPLAY VALUE		
SUPPIO	I/O DISPLAY SUPPRESSION	OUTSUPPR	
PERIOD	POINT EXECUTION PERIOD	NOPERIOD	Driven by PPS from CLM software
PVEUHI	PV EU HIGH RANGE		Max Rate Value
PVEULO	PV EU LOW RANGE		Value = 0
PVSRCOPT	PV SOURCE OPTION	ONLYAUTO	
NOPINPTS	NUMBER OF PV INPUTS	1	
PISRC(1)	PV INPUT SOURCE #1	*.PVRAW	Calculated rate from CL
PIDSTN(1)	PV INPUT DESTINATION #1	P1	For algorithm input to PV

Continued on next page

6.5.3.2 Totalizer Point (Register = Accumulated Value), Continued

Point configuration

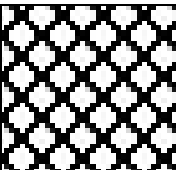
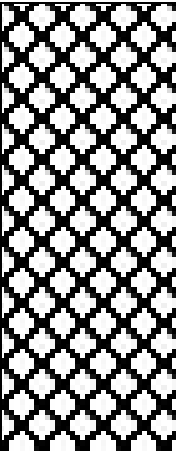
Totalizer Point Configuration (Register = accumulated value), continued

Parameter	Description	Entry	Notes
NMODE	NORMAL MODE	AUTO	
NMODATTR	NORMAL MODE ATTRIBUTE	OPERATOR	
MODEPERM	MODE PERMIT	PERMIT	
EXTSWOPT	EXTERNAL SWITCHING OPTION	NONE	
SPFORMAT	SP DECIMAL POINT FORMAT	None	This is the register “rollover” value (normally, integers would roll at 65535) Normally, 0. Not normally used. Not normally used.
SPEUHI	SP ENGINEERING UNIT HIGH		
SPEULO	SP ENGINEERING UNIT LOW		
SPHILM	SP HIGH LIMIT		
SPLOLM	SP LOW LIMIT		
SPOPT	SP OPTION		
PVHITP	PV HIGH TRIP LIMIT		
PVLOTP	PV LOW TRIP LIMIT		
CLSLOTS	NUMBER OF CL SLOTS	1	Required for attached CL
NOPKG	NUMBER OF CDS PACKAGES	2	Required to move accumulated value to SP
NOGINPTS	NUMBER OF GENERAL INPUTS	1	
PKGNAME(1)	CDS PACKAGE #1 NAME	M_TZ	Accumulated Value From accumulated value
PKGNAME(2)	CDS PACKAGE #2 NAME	CLM_DMD3	
GISRC(1)	GENERAL INPUT #1 SOURCE	*.AV	
GIDSTN(1)	GENERAL INPUT DESTINATION #1	SP	
GIACSTST(1)	GENERAL INPUT ACTIVE STS #1	ACTIVE	

Continued on next page

6.5.3.2 Totalizer Point (Register = Accumulated Value), Continued

Point configuration Totalizer Point Configuration (Register = accumulated value), continued

CDS Parameters (M_TZ) on PV Algorithm Page			
AV	REMOTE ACCUMULATION RATE		Accumulation rate value from Modbus register
PVRAW	CALCULATED RATE TO PV - PV=C1/C2 * TIMEBASE * PVRAW		Calculated by CL
C1	EU CONVERSION FACTOR		Default = 1.0
C2	EU/COUNT FACTOR		Default = 1.0
TIMEBASE	TIME BASE for PVRAW CALCS		Default = 1.0
CDS Parameters (CLM_DMD3) on Custom Page			
CLM_PPS	STORE TO PROCESS POINT PITE		Special Applications
CLM_OUT	STORE TO PROCESS OUTPUT PITE		Special Applications
DEV_STA	DEVICE IN_SERV (ON/OFF)		Updated by CLM S/W
PT_FAIL	DEVICE COMPONENT FAILURE		Not used for Modbus
HDW_REF	HARDWARE REFERENCE		Not used for Modbus
CLM_PORT	NAME OF PORT NDT		Updated by CLM Builder
CLM_DEV	NAME OF DEVICE NDT		Updated by CLM Builder
CLM_PIT	NAME OF PIT TABLE		Updated by CLM Builder

Continued on next page

6.5.3.2 Totalizer Point (Register = Accumulated Value), Continued

Builder input file – Map the image point

Accumulator Point Builder Fields

Using the CLM Builder, the specific parameters of the Accumulator type point must be mapped to the required Modbus registers.

Prompt	Entry	Description
ENTRY	M_TZ2	Template Name (.ZT)
NAME		LCN Point Name
PIT_NAME		NDT PIT Table Name
AV		Accumulation value register address
P01_AD2		Modbus data type (3 or 4)
P01_ICNV		Input conversion code
END		End of current entry

Link the CL Block

The CL block (M_TZ2CL.AO) must be linked to all Totalizer points. The script file M_TZ2CL.EC may be used to link and unlink the CL block. Otherwise, the commands are:

INACT <Totalizer Point>

UNLK M_TZ2CL <Totalizer Point> (If block is linked)

LK M_TZ2CL <Totalizer Point>

ACT <Totalizer Point>

This CL will process prior to the PV algorithm processing and place the calculated rate value into PVRAW, which is used as the P1 input to the Data Acquisition (DATAACQ) PV algorithm. The time frequency is based on the current time difference since the last update.

6.5.4 Numeric Point (AM Numeric Point Type)

Introduction

The AM Numeric point images a single register as an analog value from the Modbus device. This section describes how to create and configure this point type. Note that all parameters below are not necessarily shown unless they are required or may be utilized by the design engineer.

Theory of operation

Input and output may be accomplished for this point directly through the PV of the image point. The PV is mapped to the Modbus device register and is updated cyclically. Output may be accomplished from the PV as well with a read-back command issued after a configured period (RB_TIME).

Point configuration – Build the image point

Numeric Point Configuration

Using the LCN Data Entity Builder, an AM image point must be built, using the table following as a template for data entry. Entries left blank are either optional or user defined.

Parameter	Description	Entry	Notes
NAME	POINT NAME		
UNIT	UNIT ID		
PTDESC	POINT DESCRIPTOR		
KEYWORD	POINT KEYWORD		
ASSOCDSP	Schematic Invoked Using the “ASSOC DISP” key		
PRIMMOD	PRIMARY MODULE POINT ID	----	
EUDESC	E. U. Descriptor		
RANGEHI	PV Range Hi		
RANGELO	PV Range Lo		
PVFORMAT	PV Decimal Point Format		
PV	PV		Input from/ Output to the MB register

Continued on next page

6.5.4 Numeric Point (AM Numeric Point Type), Continued

Builder input file – Map the image point

Numeric Point Builder Fields

Using the CLM Builder, the specific parameters of the Numeric type point must be mapped to the required Modbus register

Prompt	Entry	Description
ENTRY	M_NUMER	Template Name (.ZT)
NAME		LCN Point Name
PIT_NAME		NDT PIT Table Name
PV		Modbus register address
P01_AD2		Modbus data type (3 or 4)
P01_ICNV		Input conversion code
P01_OCNV		Output conversion code
END		End of current entry

6.5.5 Digital Input (AM Switch Point Type)

Introduction

The Digital Input point uses the AM Switch point type to image one or two logical coils or status input bits from the Modbus device.

CDS parameters that are a function of the number of states are included in the separate CLM Platform software as packages DIN3CDS and DIN4CDS.

Theory of Operation

The Modbus application supports the three and four state Digital Input points as defined within the *CLM Platform Implementation Manual*.

Logical applications in Modbus device that are defined as a single on/off state (or bit) are mapped using the 3-state Digital Input.

Logical applications in the Modbus device that are defined using two bits to describe the logical state (e.g. the limit switches on a motor operated valve (MOV)) are mapped using the 4-state Digital Input.

This section contains only information for mapping the data for an application specific device.

Custom Data Segments

The Modbus Digital Input point type does not use any additional CDS packages. Only the standard CLM platform packages apply.

Point configuration – Build the image point (See Platform Manual)

Digital Input Point Configuration

Using the LCN Data Entity Builder, an AM image point must be built, using the table in Section 8.2 of the *CLM Platform Implementation Manual* for data entry.

Continued on next page

6.5.5 Digital Input (AM Switch Point Type), Continued

Builder input file – Map the image point

Digital Input Point Builder Fields

Using the CLM Builder, the specific parameters of the Digital Input type point must be mapped to the required Modbus registers.

Prompt	Entry	Description
ENTRY	M_DIN3 or M_DIN4	Template Name (.ZT) (3-state or 4-state)
NAME		LCN Point Name
PIT_NAME		NDT PIT Table Name
D1		Input #1 - Address
P01_AD2		Modbus data type (1 to 4)
P01_ICNV		Input #1 - Input conversion code
D2		Input #2 - Address (M_DIN4 only)
P02_AD2		Input #2 - Modbus data type (1 to 4) (M_DIN4 only)
P02_ICNV		Input #2 - Input conversion code (M_DIN4 only)
END		End of current entry

6.5.6 Digital Composite (AM Switch Point Type)

Introduction

The Digital Composite point uses the AM Switch point type to collect up to two logical Modbus register values and generate three logical Modbus register values in a single image point.

CDS parameters that are a function of the number of states are included in the separate CDS packages within the CLM Platform.

Theory of Operation

The Modbus application supports the three, four and five state Digital Composite points as defined within the *CLM Platform Implementation Manual*.

Logical applications in the Modbus device that use a single logical bit (1-0) for feedback are mapped using the 3-state Digital Composite. This point type will support either one or two required logical outputs.

Logical applications in the Modbus device that use two logical bits (1-0, 1-1, 0-1, 0-0) for feedback are mapped using the 4-state or 5-state Digital Composite. This point type will support either one, two or three required logical outputs.

Logical applications in the Modbus device that use two logical bits (1-0, 1-1, 0-1, 0-0) for feedback AND require three logical outputs (e.g. MOV w/STOP capability) are mapped using the 5-state Digital Composite.

This section contains only information for mapping the data for an application specific device.

Custom Data Segments

The Modbus Digital Composite point type does not use any additional CDS packages. Only the standard CLM platform packages apply.

Point configuration – Build the image point (See Platform Manual)

Digital Composite Point Configuration

Using the LCN Data Entity Builder, an AM image point must be built, using the table in Section 8.2 of the *CLM Platform Implementation Manual* for data entry.

Continued on next page

6.5.6 Digital Composite (AM Switch Point Type), Continued

**Builder input file –
Map the image point**

Digital Composite Point Builder Fields

Using the CLM Builder, the specific parameters of the Digital Composite type point must be mapped to the required Modbus registers.

Prompt	Entry	Description
ENTRY	M_D3 M_D4 M_D5	Template Name (.ZT)
NAME		LCN Point Name
PIT_NAME		NDT PIT Table Name
D1		Input #1 – Address
P01_AD2		Input #1 – Modbus data type (1 to 4)
P01_ICNV		Input #1 – Input conversion code
D2		Input #2 – Address (M_D4, M_D5 only)
P02_AD2		Input #2 – Modbus data type (1 to 4) (M_D4, M_D5 only)
P02_ICNV		Input #2 - Input conversion code (M_D4, M_D5 only)
OP1		Output #1 – Address
FB1		Feedback #1 –Address
P04_AD2		Feedback #1 – Modbus data type (1 to 4)
P04_ICNV		Feedback #1 – Input conversion code
OP2		Output #2 – Address (M_D4, M_D5 only)
FB2		Feedback #2 – Address (M_D4, M_D5 only)
P04_AD2		Feedback #2 – Modbus data type (1 to 4) (M_D4, M_D5 only)
P04_ICNV		Feedback #2 – Input conversion code (M_D4, M_D5 only)
OP3		Output #3 – Address (M_D4, M_D5 only)
FB3		Feedback #3 – Address (M_D4, M_D5 only)
P08_AD2		Feedback #3 - Modbus data type (1 to 4)
P08_ICNV		Feedback #3 – Input conversion code
END		End of current entry

6.5.7 Flag Point (AM Flag Point Type)

Introduction

The AM Flag point images a single coil or Input Status bit (read-only) as a logical value from the Modbus device. This section describes how to create and configure this point type. Note that all parameters below are not necessarily shown unless they are required or may be utilized by the design engineer.

Theory of operation

The Flag point images a value in the Modbus device that is either logically “ON” or “OFF”. The PV of the Flag point is mapped to that logical value. The Flag point can be configured to output a change in logical value or be used as a read-only indication. Logical inversion of the value may also be configured in the conversion codes.

Point configuration – Build the image point

Flag Point Configuration

Using the LCN Data Entity Builder, an AM image point must be built, using the table following as a template for data entry. Entries left blank are either optional or user defined.

Parameter	Description	Entry	Notes
NAME	POINT NAME	--	
UNIT	UNIT ID		
PTDESC	POINT DESCRIPTOR		
KEYWORD	POINT KEYWORD		
PRIMMOD	PRIMARY MODULE POINT ID		
PVSTATES0	UPPER BOX TEXT		
PVSTATES1	LOWER BOX TEXT		
UBOXCLR	UPPER BOX COLOR		
LBOXCLR	LOWER BOX COLOR		
OFFNRMAL	OFF NORMAL ALARMING		
PVNORMAL	PV NORMAL STATE	OFF ON	Only if OFFNRMAL=ON

Continued on next page

6.5.7 Flag Point (AM Flag Point Type), Continued

Builder input file – Map the image point

Flag Point Builder Fields

Using the CLM Builder, the specific parameters of the Flag type point must be mapped to the required Modbus registers.

Prompt	Entry	Description
ENTRY	M_FLAG	Template Name (.ZT)
NAME		LCN Point Name
PIT_NAME		NDT PIT Table Name
PV		Coil Address
P01_AD2		Modbus data type (1 to 4)
P01_ICNV		Input conversion code
P01_OCNV		Output conversion code (Modbus data type 1 only)
END		End of current entry

6.5.8 Message Point (AM Custom Point Type)

Introduction

The message point uses the AM custom point type to image up to twenty consecutive Modbus registers as a string. This section describes how to create and configure this point type. Note that all parameters below are not necessarily shown unless they are required or may be utilized by the design engineer.

Theory of operation

This point manages a list of the last 20 messages sent or received using a CDS array. Message length is limited to 40 characters. A CL block is attached to the Message point to manipulate the data as it comes in or goes out. There is an option to set a CDS logical parameter to journal a message for every message string that comes in or goes out. The CL also keeps the time and date of each message (input or output).

Special Graphic Interface available

Message Point Display. Data for the Message point can be easily displayed by use of a special graphic provided with the Modbus package (CLMMODB0.DS). This display prompts the user for the Message image point name and then displays the last twenty messages either input or output along with the time and date of each message. There is an operator interactive function to send a message from the display. Refer to Section 7 of this manual for further details.

Continued on next page

6.5.8 Message Point (AM Custom Point Type), Continued

Point configuration – Build the image point

Message Point Configuration

Using the LCN Data Entity Builder, an AM image point must be built, using the table following as a template for data entry. Entries left blank are either optional or user defined.

Parameter	Description	Entry	Notes
NAME	POINT NAME		
UNIT	UNIT ID		
PTDESC	POINT DESCRIPTOR		
KEYWORD	POINT KEYWORD		
CLSLOTS	NUMBER OF CL BLOCKS	1	
NOPKG	No. OF CUSTOM DATA PACKAGES	2	
PRIMMOD	PRIMARY MODULE POINT ID	--	
ALPRIOR	ALARM PRIORITY		
ALENBST	ALARM STATE		
PKGNAME(1)	SOURCE FILE 1 NAME	M_MSG	
PKGNAME(2)	SOURCE FILE 2 NAME	CLM_DMD3	
PERIOD	POINT EXECUTION PERIOD	NOPERIOD	

Continued on next page

6.5.8 Message Point AM Custom Point Type), Continued

CDS parameters CDS parameters from two packages can appear on the custom detail pages.

Message Point CDS Parameters

Parameter	Description	Entry	Notes
CDS Parameters (M_MSG) on Custom Page			
INPUT	INPUT STRING		Message data received
OUTPUT	ENTERED OUTPUT STRING		Message data sent
LAST_IN	LAST INPUT STRING		
LAST_OUT	LAST OUTPUT STRING		
IOARRAY	I/O STRINGS		Array of last 20 messages
IOTIME	I/O STRING DATE/TIME		Array of date/time for the IOARRAY.
OUTFLAG	I/O STRING WAS OUTPUT		Logical array for the IOARRAY. Set true for output strings
JOURNAL	OPERATOR JOURNAL OPTION		OFF ON (Build Visible)
ON_DISP	MESSAGE POINT ON DISPLAY		US Node Number
CDS Parameters (CLM_DMD3) on Custom Page			
CLM_PPS	STORE TO PROCESS POINT PITE		Special Applications
CLM_OUT	STORE TO PROCESS OUTPUT PITE		Special Applications
DEV_STA	DEVICE IN_SERV (ON/OFF)		Updated by CLM S/W
PT_FAIL	DEVICE COMPONENT FAILURE		Not used for Modbus
HDW_REF	HARDWARE REFERENCE		Not used for Modbus
CLM_PORT	NAME OF PORT NDT		Updated by CLM Builder
CLM_DEV	NAME OF DEVICE NDT		Updated by CLM Builder
CLM_PIT	NAME OF PIT TABLE		Updated by CLM Builder

Continued on next page

6.5.8 Message Point (AM Custom Point Type), Continued

Builder input file – Map the image point

Message Point Builder Fields

Using the CLM Builder, the specific parameters of the Message type point must be mapped to the required Modbus registers.

Prompt	Entry	Description
ENTRY	M_MSG	Template Name (.ZT)
NAME		LCN Point Name
PIT_NAME		NDT PIT Table Name
INPUT		Register address for Input String
P01_AD2		Modbus data type (3 or 4)
P01_ICNV		Input conversion code for input string
OUTPUT		Register address for Output String
P02_OCNV		Output conversion code for output string
END		End of current entry

Link the CL Blocks

The Message point requires a CL block to be linked. This requires the point to then be set active for proper execution.

Block Name	Insertion Point	Insertion Order	Description
M_MSGCL	GENERAL	10	CL message handling function

Note: The file M_MSG.EC may be used to unlink these CL blocks and link the package M_MSGCL.AO.

Continued on next page

6.5.9 Array Input / Output (AM Custom Point Type), Continued

Introduction

The CLM Modbus Application supports fetching and storing of logical and analog arrays.

Since array points are application specific, the construction and naming of CDS parameters and the development of the TDFs are left for the user.

Hint! Sample array input TDFs and array output TDFs are included with the CLM Modbus Application files and should be available for modification and use.

Theory of operation

The array point is generally configured as an AM Custom point type with a user-generated CDS package attached. This package contains a defined parameter array of numeric or logical values. A single array parameter can accommodate a maximum of 1000 elements.

The user must create a Template Definition File (.ZT) to map the array parameter. The sample files included with the release package may be used as a starting point or example.

In the template file, the user must define the name of the array parameter, the data item address, and the conversion code that indicates the type and size of the array data.

In the PITE conversion code, the LCN code used specifies whether the array is logical or numeric, and the number of elements in the array. See Section 5.3 for detailed information.

The basic format of the conversion code for the array parameter is

Logical Array: $8,n-f,o$
 where n = number of elements (1..1000)
 f = Modbus function code
 o = Conversion options

Numeric Array: $7,n-f,o$
 where n = number of elements (1..1000)
 f = Modbus function code
 o = Conversion options

Continued on next page

6.5.9 Array Input / Output (AM Custom Point Type), Continued

Point Configuration – Build the image point

Array Point Configuration.

Using the LCN Data Entity Builder, an AM image point must be built, using the table following as a template for data entry. Entries left blank are either optional or user defined.

Parameter	Description	Entry	Notes
NAME	POINT NAME		
UNIT	UNIT ID		
PTDESC	POINT DESCRIPTOR		
KEYWORD	POINT KEYWORD		
CLSLOTS	NUMBER OF CL BLOCKS	<As Required>	Optional, by user
NOPKG	No. OF CUSTOM DATA PACKAGES	2	
PRIMMOD	PRIMARY MODULE POINT ID	--	
ALENBST	ALARM STATE	ENABLE	
PKGNAME(1)	SOURCE FILE 1 NAME	<User Defined>	Contains the array parameter(s)
PKGNAME(2)	SOURCE FILE 2 NAME	CLM_DMD3	
PERIOD	POINT EXECUTION PERIOD	NOPERIOD	Triggered by CLM s/w
CDS Parameters (Defined by User, listed here generically)			
INP_ARR	INPUT ARRAY		From Modbus register
OUT_ARR	OUTPUT ARRAY		To Modbus register
CLM_DMD3 CDS Parameters on Custom Page			
CLM_PPS	STORE TO PROCESS POINT PITE		Special Applications
CLM_OUT	STORE TO PROCESS OUTPUT PITE		Special Applications
DEV_STA	DEVICE IN_SERV (ON/OFF)		Updated by CLM S/W
PT_FAIL	DEVICE COMPONENT FAILURE		Not used for Modbus
HDW_REF	HARDWARE REFERENCE		Not used for Modbus
CLM_PORT	NAME OF PORT NDT		Updated by CLM Builder
CLM_DEV	NAME OF DEVICE NDT		Updated by CLM Builder
CLM_PIT	NAME OF PIT TABLE		Updated by CLM Builder

Continued on next page

6.5.9 Array Input / Output (AM Custom Point Type), Continued

Builder input file:
Array Input -
Map the array

Array Input Builder Fields (Created by User)

The fields below define the minimum entries required by the User-created file. These user-visible fields **MUST** correspond exactly to the template file (.ZT) created by the User.

Using the CLM Builder, the specific parameters of the Array Input type point must be mapped to the required Modbus registers.

Prompt	Entry	Description
ENTRY	M_ARYI	Template Name (or user defined)
NAME		LCN Point Name
PIT_NAME		NDT PIT Table Name
ARY_NAM		Name of the user defined numeric array parameter in a CDS attached to the LCN Point
ARY_ADR		Starting address of the contiguous data in the Modbus device
ARY_AD2		Modbus Data type (1 to 4)
ARY_ICNV		Array input conversion code – See Section 6.3
END		End of current entry

Builder input file:
Array Output -
Map the array

Array Output Builder Fields (Created by User)

The fields below define the minimum entries required by the User-created file. These user-visible fields **MUST** correspond exactly to the template file (.ZT) created by the User.

Using the CLM Builder, the specific parameters of the Array Output type point must be mapped to the required Modbus registers.

Prompt	Entry	Description
ENTRY	M_ARYO	Template Name (or user defined)
NAME		LCN Point Name
PIT_NAME		NDT PIT Table Name
ARY_NAM		Name of the user defined logical array parameter in a CDS attached to the LCN Point.
ARY_ADR		Starting address of the contiguous data in the Modbus device
ARY_AD2		Modbus Data type (1 or 3 only)
ARY_OCNV		Array output conversion code – See Section 6.3
END		End of current entry

Continued on next page

6.5.9 Array Input / Output (AM Custom Point Type), Continued

Array Input – Example

User needs to read a data array from a device that has 1500 elements.

Hint! Since the data array to be read is larger than the 1000 element maximum, create two array parameters.

User creates two array parameters, ARR1 and ARR2 in a CDS package and attaches to a Custom AM point. Each array parameter is defined to contain 750 elements.

User then creates a template file (.ZT) to map the two array parameters. Specific fields within the template file will be defined as follow below:

```
NUM_PAR = 2
ARRAY1NAM = ARR1
ARRAY1ADR = 1
P01_ICNV = 7,750-3,3
ARRAY2NAM = ARR2
ARRAY2ADR = 751
P02_ICNV = 7,750-3,3
```

Reminder! The name given to the template file (.ZT) must be unique within the application environment AND is the same as the ENTRY NAME field.

Section 7 – Modbus Image Point Operation

7.1 Overview

Introduction

Many of the supported image point types in the Modbus application require operator interaction to either send or receive data.

This section describes those actions that directly impact the sending or receiving of data from the Modbus network.

This section **does not** describe those actions which would be a normal part of the TDC image point , such as changing trip limits or ranges, placing a point active or inactive, or executing a PPS of a point.

Note: All actions described in the following sections assume an In-Service CLM/Modbus network and scheduled PITs driving the image points where applicable.

7.2 Operation – Analog Input

Starting the Point

There are no special actions required to receive data from the Modbus device other than placing the point execution state (PTEXECST) to Active.

It should be noted that the CLM will place data into the INPUT CDS parameter on the point, regardless of the point execution state. Placing the point execution state to Active allows for the PV algorithm to execute, pushing the value to the PV.

Configuration Actions

None, other than normal trip limit and range configuration.

Operational Actions

None.

7.3 Operation – Analog Output

Starting the Point

The point execution state must be set Active to allow outputs from the SP. Also, because it is the SP, the point must be placed in AUTO.

Configuration Actions

- The SPEULO and SPEUHI must be set to the output value range.
 - The CVEULO and CVEUHI must be set equal to the SPEULO and SPEUHI.
 - The point must be in AUTO mode.
 - The SPILIM and SPLOLIM should be set equal to the SPEULO and SPEUHI.
 - The OPEULO and OPEUHI is in percent and should be set to 0 and 100.
-

Send an Output

Send an analog value to the Modbus device register:

- Be sure that the point is in AUTO.
 - Select the SP on the Detail display, or press the SP button on the Operator console.
 - Enter the value to be sent to the Modbus register.
 - Press ENTER.
 - Wait for update from the readback of the register and verify the correct range value is generated in the OP.
-

7.4 Operation – Accumulator (Register = Rate value)

Starting the Point

The point execution state of the point must be set to Active for the accumulation process. This accumulator type uses the Totalizer algorithm on the AM Regulatory point and processes based on the rate value being input from the Modbus register.

Configuration Actions

- A value must be entered into the C (Scaling) parameter. Use 1 for a default value.
 - A value must be entered into the RESETVAL when restarting the accumulation.
 - The PVEULO and PVEUHI must be set to zero and the maximum accumulation value.
-

Set AVTV

Set an Accumulation Target Value:

- From the Detail display, select the SP (AVTV) or press the SP button on the Operator console.
 - Enter the Accumulation target value.
 - Press ENTER.
-

Set AVDEV1TP

Set a coarse Deviation trip value for the accumulation:

- Determine the coarse deviation between the AVTV and the PV that needs to be flagged.
 - From the Detail display, select the AVDEV1TP.
 - Enter the coarse deviation value.
 - Press ENTER.
 - When the absolute difference between the AVTV and the PV is less than or equal to the AVDEV1TP, the AVDEV1FL parameter flag on the point will be set to ON. This flag can be used to trigger CL or as an input to another point for alarming.
-

Continued on next page

7.4 Operation – Accumulator (Register = Rate value),

Continued

Set AVDEV2TP

Set a fine Deviation trip value for the accumulation:

- Determine the fine deviation between the AVTV and the PV that needs to be flagged.
- From the Detail display, select the AVDEV2TP.
- Enter the fine deviation value.
- Press ENTER.
- When the absolute difference between the AVTV and the PV is less than or equal to the AVDEV2TP, the AVDEV2FL parameter flag on the point will be set to ON. This flag can be used to trigger CL or as an input to another point for alarming.

When AVTV = PV

Take action when the Accumulated value reaches the Target value:

- When the PV is greater than or equal to the AVTV, then the AVTVFL parameter flag is set to ON. This value may be used to trigger CL or as an input to another point for alarming.

Start Accumulation

Start the Accumulation Process:

- From the Detail display, select the Mode value at the bottom of the display (should show STOPPED, under the OP value).
 - From the Change Zone values, select RESET.
 - Press ENTER.
 - Confirm that the RESETVAL is placed into the PV.
 - Again, select the Mode value at the bottom of the display.
 - From the Change Zone values, select START.
 - Press ENTER.
 - Verify that the accumulation value (PV) is incrementing as expected and the state now shows RUNNING.
-

Continued on next page

7.4 Operation – Accumulator (Register = Rate value),

Continued

Stop Accumulation

Stop the Accumulation Process:

- From the Detail display, select the Mode value at the bottom of the display (should show RUNNING, under the OP value).
 - From the Change Zone values, select STOP.
 - Press ENTER.
 - Verify that the accumulation value (PV) has stopped incrementing and the state now show STOPPED.
-

7.5 Operation – Accumulator (Register = Accumulated value)

Starting the Point

The point execution state of the point must be set to Active. This accumulator type uses CL to determine a rate based on an accumulation value being input from the Modbus register.

Configuration Actions

- A value must be entered into the C (Scaling) CDS parameter. Use 1 for a default value.
 - The PVEULO and PVEUHI must be set to zero and the maximum accumulation rate value.
 - The SPEULO and SPEUHI must be set to 0 and the maximum accumulation value (or the rollover value).
 - The SPLOLIM and SPHILIM must be set to the SPEULO and SPEUHI.
-

Operational Actions

None.

7.6 Operation – Numeric

Starting the Point

None. There is no PPS and no CDS may be attached.

Configuration Actions

- The PVEULO and PVEUHI may be configured for required ranging.
-

Send an Output

Send an output from the PV:

- From the Detail display, select the PV.
 - Enter the new analog value.
 - Press ENTER.
-

7.7 Operation – Digital Input

Starting the Point	Refer to the <i>CLM Platform Implementation Manual</i> . This point must set the point execution state to Active since a CL block is linked to this point type.
Configuration Actions	Refer to the <i>CLM Platform Implementation Manual</i> .
Set Tracking	Set the point to have the S1REQSTS track the S1CURSTS: • From the Detail display, select the S2REQSTS. • From the Change Zone, select TRACK. • Press ENTER.
Set No Tracking	Set the point to have the S1REQSTS <i>NOT</i> track the S1CURSTS: • From the Detail display, select the S2REQSTS. • From the Change Zone, select NOTRACK. • Press ENTER.
Operational Actions	None.

7.8 Operation – Digital Composite

Starting the Point

Refer to the *CLM Platform Implementation Manual*. This point must set the point execution state to Active since a CL block is linked to this point type.

Configuration Actions

Refer to the *CLM Platform Implementation Manual*.

Clear the Point

To clear all current alarms on the point and set the S1REQSTS = S1CURSTS:

- From the Detail display, select the S2REQSTS parameter.
 - From the Change Zone, select the CLEAR option.
 - Press ENTER.
-

Change the State

To change the state of the Digital Composite point:

- From the Detail display, select the S1REQSTS.
 - From the Change Zone, select the new state value.
 - Press ENTER.
 - Verify that the input bit(s) change as expected and no alarms are generated on the point.
-

7.9 Operation – Flag

Starting the Point

This point has no PPS and cannot have attached CDS.

Configuration Actions

Off Normal alarming may be configured on the Flag point.

Set Off Normal Alarming

To set Off Normal alarming on the Flag point:

- From the Detail display of the point, select the OFFNRMAL parameter.
 - From the Chang Zone, select the ON option.
 - Press ENTER.
 - Select the PVNORMAL parameter.
 - From the Change Zone, select the state that will be the normal, or non-alarm state.
 - Press ENTER.
-

Change the State

To change the logical state of the mapped Modbus device register:

- From the Detail display, select the PV.
 - Press the “State-Up” button on the Operator console.
 - The opposite state appears above the current PV state.
 - Press ENTER.
 - The new state is sent to the Modbus register and reflected on the point.
-

7.10 Operation - Message Point

Message Points

The message point, Section 6.5.8, allows several consecutive registers to be treated as a string. If you require the entry of strings longer than 16 characters, you must use the display described in this section.

Starting the Point

This point must have the point execution state set to Active and the CL linked.

Configuration Actions

None.

Message Point Display Structure

The message point display (CLMMODB0.DS) is independent of other displays. The first page, Figure 7.10-1, has an input port for entering the tag name of a Modbus message point. The second display page, Figure 7.10-2, resembles a detail page. In these displays, the tag name M_MSG can be any valid message point tag name that you have built.

Accessing the Message Point Display

Accessing the Message Point Display

Step	Activity
1	Press <SCHEM> on the operator's keyboard.
2	In response to the prompt "Enter Schematic Name", key in CLMMODB0 and press <ENTER>, The system responds with the initial Message Point display - Figure 7.10-1.
3	On the initial Message Point display, enter the tag name of the requested message point. The system responds with the display shown in Figure 7.10-2.
4	View the data for the last 20 input or output strings
5	To send a string, pick the OUTPUT MESSAGE target to process the point and transmit the string.
6	After the string has been transmitted, the display updates with your output string at the top of the message list. All other lines move down one position.

Continued on next page

7.10 Operation - Message Point, Continued

Figure 7.10-1 Enter Message Point ID Display

ENTER MESSAGE POINT ID

11 DEC 96 16:18:55 2

M_MSG

Figure 7.10-2 Message Point Display

11 DEC 96 16:18:55 2

M_MSG description 40 UNIT H5

MESSAGE POINT DATA

LAST/IN INPUT STRING

LAST/OUT

DATE/TIME MESSAGE

01-01-96 00:00:00 IOARRAY 1

01-01-96 00:00:00 IOARRAY 2

01-01-96 00:00:00 IOARRAY 3

01-01-96 00:00:00 IOARRAY 4

OUTPUT MESSAGE

Message Point Display

The message point display shows an array of the previous 20 messages (inputs and outputs). The input strings are shown in cyan and the output strings in white.

Section 8 – References

8.1 Reference Documents

Reference documents

- *Modicon Modbus Protocol Reference Guide*, Document P1-MBUS-300-Rev. D, March, 1992.
 - *IEEE Standard for Binary Floating-Point Arithmetic*, ANSI/IEEE Std 754-1985.
 - David V. James, "Multiplexed Busses: The Endian Wars Continue", *IEEE Micro*, Vol. 10 No. 3, June 1990, pp. 9-21.
 - *CLM Platform Implementation Manual*, Revision, Revision 5, October, 1999.
-

Appendix A – NDT – Network Definition File Templates

A.1 Understanding NDT Template Definition Files

Explanation of NDT TDF Lines

Initial NDT TDF Lines

PITE Field Name	Visible	Description
ENTRY	Yes	Name of the template file
TBL_TYPE		Defined table type (NDT)
ENT_SIZ		System generated: The size of this NDT entry
NAME	Yes	Name used in network memory for this NDT entry
ENT_NUM		System generated: Revision code and internal table ID
NUM_KEYS		Number of key parameters
A1_CLMI	Yes	CLMI number, 1 or 2
A2_PORT	Yes/No	Port Number, 1 to 4 (Not visible on the CLMI NDT)
A3_DEV	Yes/No	Device, 0 to 247 (Not visible on CLMI and PORT NDTs) Note: Device 0 is used for broadcast.
A4 A5 A6 A7 A8		Not used for Modbus
OPT UPLOAD DOWNLOAD RBE	No No	Upload and download are not visible for Modbus devices Upload device (Y/N) Download device data (Y/N) RBE is not used for Modbus
UP_STS DWN_STS		Status on last load operations: Visible on the appropriate NDT status display. (Not used for Modbus.)

Continued on next page

A.1 Understanding NDT Template Definition Files, Continued

Initial NDT TDF Lines, Continued

PITE Field Name	Visible	Description												
FAIL_TM		Time to wait before changing operating state to FAILED												
FAIL_AT		System saves time of last failure												
STS		High level operating status of this NDT component: <table border="1"> <tr> <td>0</td><td>GOOD</td><td>The parameter input was successful</td></tr> <tr> <td>1</td><td>ERROR</td><td>The parameter input was successful, but the value may be questionable</td></tr> <tr> <td>2</td><td>FAILED</td><td>The device is available but the parameter is not available.</td></tr> <tr> <td>3</td><td>UNKNOWN</td><td>The device is unavailable</td></tr> </table>	0	GOOD	The parameter input was successful	1	ERROR	The parameter input was successful, but the value may be questionable	2	FAILED	The device is available but the parameter is not available.	3	UNKNOWN	The device is unavailable
0	GOOD	The parameter input was successful												
1	ERROR	The parameter input was successful, but the value may be questionable												
2	FAILED	The device is available but the parameter is not available.												
3	UNKNOWN	The device is unavailable												
DET_STS		The detailed status array. Refer to CLM Platform Manual..												
STA		Operating state of this NDT component: <table border="1"> <tr> <td>0</td><td>IN_SERV</td><td>The component is in service.</td></tr> <tr> <td>1</td><td>SYNC</td><td>The component is in service for database synchronization only.</td></tr> <tr> <td>2</td><td>FAILED</td><td>The component has failed and the FAIL_TM timer has expired. The user must reset by taking the component to OUT_SERV and then to IN_SERV.</td></tr> <tr> <td>3</td><td>OUT_SERV</td><td>The component has been set out of service by an operator.</td></tr> </table>	0	IN_SERV	The component is in service.	1	SYNC	The component is in service for database synchronization only.	2	FAILED	The component has failed and the FAIL_TM timer has expired. The user must reset by taking the component to OUT_SERV and then to IN_SERV.	3	OUT_SERV	The component has been set out of service by an operator.
0	IN_SERV	The component is in service.												
1	SYNC	The component is in service for database synchronization only.												
2	FAILED	The component has failed and the FAIL_TM timer has expired. The user must reset by taking the component to OUT_SERV and then to IN_SERV.												
3	OUT_SERV	The component has been set out of service by an operator.												

Key parameters

Two-Line Key Parameters

size	Key size is entered on the first line. Enter the number of words allocated for the key parameter.
name	Enter the parameter name first on the second line.
visibility	The visibility code can be : or =. Equal makes the parameter user-visible.
format	Format codes are defined in the <i>CLM Platform Implementation Manual</i> .
range	Range values optionally follow the I or integer format code.
default value	Following the format, or the range if present, is a slash. The optional default value follows the slash.

Appendix B – PITE Template Definition Files

B.1 Point Interface Templates

TDF files

Each of the TDF files has a ZT extension. For example, the name of the file containing the analog input point template is M_AI.ZT.. These files are installed in the NET>&CUS directory during the Modbus installation and can be reviewed for content. Ref. Table 3.3-1 for the list of templates.

Explanation of PITE TDF Lines

The following is the PITE template file for the Modbus analog input point.

PITE Field Name	Visible	Description
ENTRY	Yes	Name of the template file
TBL_TYPE		Defined table type (PITE)
ENT_SIZ		The system-generated size of this PIT entry
NAME	Yes	AM image point name used by this PIT entry
PT_ID		System generated: The internal point code used by data access
PIT_NAME	Yes	The name of the PIT table to which this PITE belongs
DPOINT		Not used for Modbus
DPT_AD1		Not used for Modbus
DPT_AD2		Not used for Modbus
NDT_PO		System-generated page and offset pointer to NDT entry for the device
RB_TIME	No	Number of seconds to wait before reading back an output value if the read back option (RB) is selected for any PITE parameter
PT_OPT [PPS, CHG_ONLY]	Yes	PITE options: PPS: Issue a PPS command to process the AM image point after processing this PITE. CHG_ONLY: Only store changed values from the register to the image point parameter.
NUM_KEYS		Number of key parameters (see below): Must be correct
NUM_PARS		Number of point parameters: Must be correct

Parameter Addressing

The user may enter Modbus address and conversion strings for each PITE parameter. The conversion strings are entered as ICNV (input) and OCNV (output). Refer to Section 6.3 for details.

Continued on next page

B.1 Point Interface Templates, Continued

Explanation of PITE TDF Lines

Note: The user may rename any of the visible parameters in this table. Builder constraints as defined in the *CLM Platform Implementation Manual* must be observed.

TDF Lines Repeated for Each PITE Parameter

PITE Field Name	Visible	Description												
Pnn_PAR	No	AM image parameter for which this entry is defined												
Pnn_ID	No	System-generated internal parameter identification												
Pnn_QUAL	No	System-generated internal parameter qualifier code												
Pnn_DPAR	No	Not used by Modbus												
Pnn_AD1	Yes	Integer parameter register or coil address in the Modbus device, 0001 to 10000: This parameter is usually renamed to reflect an actual signal name.												
Pnn_AD2	Yes	Enter Modbus data type: <table border="1"> <tr> <td>1</td><td>Coil</td><td>Use for discrete values</td></tr> <tr> <td>2</td><td>Input status</td><td>Use for discrete values</td></tr> <tr> <td>3</td><td>Holding register</td><td>Use for analog values and strings</td></tr> <tr> <td>4</td><td>Input register</td><td>Use for analog values and strings</td></tr> </table>	1	Coil	Use for discrete values	2	Input status	Use for discrete values	3	Holding register	Use for analog values and strings	4	Input register	Use for analog values and strings
1	Coil	Use for discrete values												
2	Input status	Use for discrete values												
3	Holding register	Use for analog values and strings												
4	Input register	Use for analog values and strings												
Pnn_ISTA	No	System variable: State of the input type_value_record												
Pnn_IFNC	No	Refer to the following individual items:												
CYC		Y= normal cycle processing, scheduled by PIT scheduler												
SLO		Y=slow cycle (based on SLO_DWN value)												
ECP		Set to N for Modbus inputs.												
PPS		Y=when point is PPS,d, update the input data.												
SYN		Y=input this parameter to synchronize the database when device is placed in service.												
CN		Y=change notification alarming: Output a journal message when the value changes.												
SB		Y=set this analog input parameter value bad when the device becomes unavailable for input due to lost communication with the CLMI.												
RBI		Y=read this input RB_TIME after a PITE parameter output with RB=Y.												
Pnn_ICNV	Yes	Input parameter conversion code: Refer Section 6.3.												

Continued on next page

B.1 Point Interface Templates, Continued

TDF Lines Repeated for Each PITE Parameter, continued

PITE Field Name	Visible	Description												
Pnn_IDST	No	Last device access status of the input parameter: Visible on the PITE detail display: <table> <tr> <td>0</td><td>GOOD</td><td>The parameter input was successful.</td></tr> <tr> <td>1</td><td>ERROR</td><td>The parameter input was successful, but the value may be questionable.</td></tr> <tr> <td>2</td><td>FAILED</td><td>The device is available but the parameter is not available.</td></tr> <tr> <td>3</td><td>UNKNOWN</td><td>The device is unavailable.</td></tr> </table>	0	GOOD	The parameter input was successful.	1	ERROR	The parameter input was successful, but the value may be questionable.	2	FAILED	The device is available but the parameter is not available.	3	UNKNOWN	The device is unavailable.
0	GOOD	The parameter input was successful.												
1	ERROR	The parameter input was successful, but the value may be questionable.												
2	FAILED	The device is available but the parameter is not available.												
3	UNKNOWN	The device is unavailable.												
Pnn_ITVO	No	System generated: Offset from this word to the input type_value_record												
Pnn_OSTA	No	System variable: State of the output type_value_record												
Pnn_OFNC	No	Refer to the following individual items:												
CYC		Y=normal cyclic processing, by PIT scheduler												
SLO		Y=slow cyclic processing (based on SLO_DWN value)												
ECP		Y=exception processing: Output is via the store change function.												
PPS		Y=output this parameter if point is PPS'd by operator.												
SYN		Y=output this parameter to synchronize the database when device is placed in service.												
RB		Y=force read back of RBI parameters after the exception output of this parameter												
CBO	No	Check Before Operate: Does not apply to Modbus												
Pnn_OCNV	Varies	Output parameter conversion code: Refer to Section 6.3.												
Pnn_ODST	No	Last device access status of the output parameter: Visible on the PITE detail display: <table> <tr> <td>0</td><td>GOOD</td><td>The parameter output was successful</td></tr> <tr> <td>1</td><td>ERROR</td><td>The parameter output was successful, but the value may be questionable</td></tr> <tr> <td>2</td><td>FAILED</td><td>The device is available but the parameter was not accepted.</td></tr> <tr> <td>3</td><td>UNKNOWN</td><td>The device is unavailable</td></tr> </table>	0	GOOD	The parameter output was successful	1	ERROR	The parameter output was successful, but the value may be questionable	2	FAILED	The device is available but the parameter was not accepted.	3	UNKNOWN	The device is unavailable
0	GOOD	The parameter output was successful												
1	ERROR	The parameter output was successful, but the value may be questionable												
2	FAILED	The device is available but the parameter was not accepted.												
3	UNKNOWN	The device is unavailable												
Pnn_OTVO	No	System-generated: Offset from this word to the output type_value_record												

Appendix C – CL Interface to Non-Honeywell Device

The *CLM Platform Implementation Manual* defines the method used to interface with Non-Honeywell Devices, specifically to send user created messages to the Modbus Application executing on the CLMI Board for transmission out of the CLMI Port. Please reference this manual for information relating to this task.

CL calls are utilized to support the implementation:

- CALL CLM\$RES_BUFFER
- CALL CLM\$REL_BUFFER
- CALL CLM\$PUT_FIELD
- CALL CLM\$GET_FIELD
- CALL CLM\$SEND_CL_MESSAGE
- CALL CLM\$RCV_CL_MESSAGE

Note: The Modbus application provides two different methods for handling the CL messages.

1) If the first character of the CL message is not equal to 248, the message will follow the normal Modbus protocol conventions (e.g. checksum, timing). This method could be used on a Modbus network to transmit a Modbus supported message.

2) If the first character of the CL message is equal to 248, the message is transmitted as received less the first character (which was 248). The returned ASCII string is returned to the CL interface for user retrieval. This method cannot be used on a Modbus network (this does not use the Modbus timing and checksum). The intent is to allow the user of the CL interface the flexibility to create a unique protocol using the MODBUS application.

Appendix D – Error Messages / Function Codes

D.1 Modbus Function Codes

The following Modbus function codes are supported:

- 01 - Read Coil(s) status
- 02 - Read Input(s) status
- 03 - Read Holding Register
- 04 - Read Input Register(s)
- 05 - Write Single Coil
- 06 - Write Single Register (Holding)
- 15 - Write Multiple Coils
- 16 - Write Multiple Registers

All function codes are available to application CL programs using the CL send/receive message interface.

D.2 Modbus Error Codes

The CLM Modbus protocol will recognize the following Modbus defined EXCEPTION RESPONSES:

- 01 - Illegal Function
 - 02 - Illegal Data Address
 - 03 - Illegal Data Value
 - 04 - Failure in Device
 - 06 - Busy
 - 07 - Negative Acknowledge
 - 08 – FF are reserved
-

D.3 CLM Platform / Modbus Error Codes

The *CLM Process Operations Manual*, Appendix A, lists additional error codes and operational messages for the CLM platform. In addition, Nonrecoverable Modbus Error Codes are also listed.

Appendix E – UCLM: CCP AND UCIM Redesigned

CCP (51402426) and UCIM (51402436) boards have been redesigned and are now known as UCLM-51306409. UCLM is a combination of CCP and UCIM.

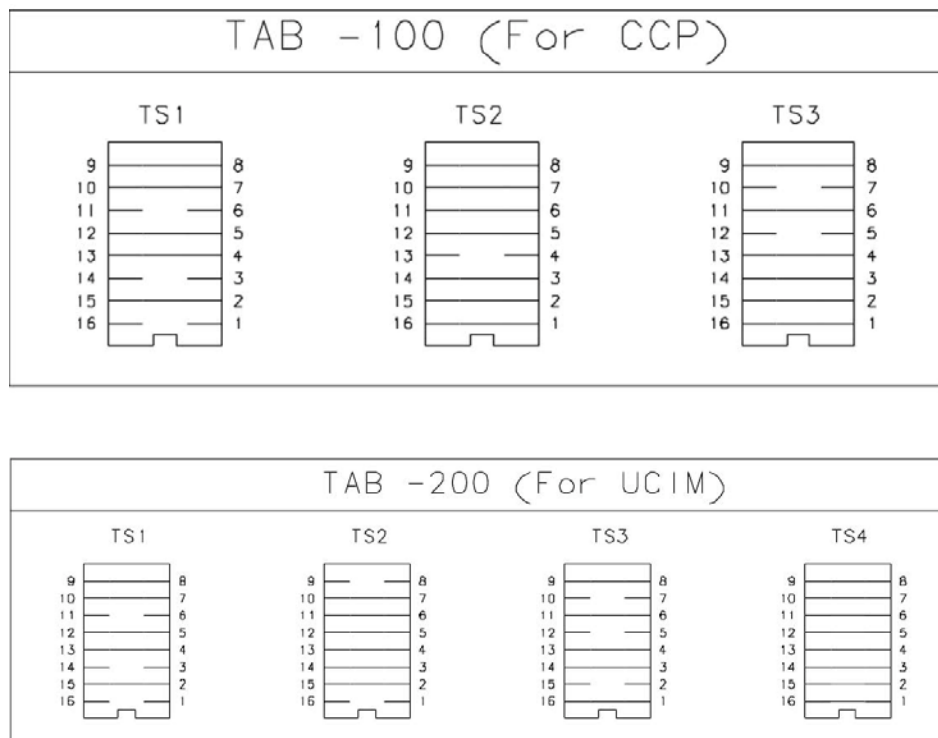
E.1 Ordering Information

To order the boards use the Honeywell part numbers as provided in the table below:

Part Number	Board Name
51306409-100	CCP
51306409-200	UCIM

E.2 Switch settings of UCLM board

Figure E-1 illustrates the redesigned CCP and UCIM settings



There is no change in the procedure to load AM personality on UCLM. Procedure remains the same as it was in CCP and UCIM.

Index

&

&CUS	
Created during AM Installation.....	41
TDF files.....	135

A

A1_CLMI.....	133
A2_PORT	133
A3_DEV	133
Accumulator Point	
Determining Type.....	90
AM	xi
CLM is a modified AM	14
APP.....	xi

B

BAUD.....	56
BIF.....	xi
Accumulator Point.....	95
Analog Input Point.....	85
Analog Output Point.....	89
Array Input Point.....	114
Array Output Point	114
Digital Composite Point.....	105
Digital Input Point.....	103
File creation	52
Flag Point.....	107
Message Point	111
Numeric Point	101
Totalizer Point.....	99

C

CBO.....	xi
CCP	xi
CDS	xi
CHG_ONLY	135
CLxi	
Accumulator Point.....	95
Interface to non-Honeywell.....	139, 143
Message Point	111
Totalizer Point.....	99
CLM	xi
Configuration Overview	51
CLM layers	
Communication protocol layer	17

Four layers	17
Network interface layer	17
Platform layer	15, 17
Process application layer	17

CLMI.....	xi
Slot numbers.....	46

CLMI/CCP

Configuration file,.(ZC)	44
Compile CLM Pictures	43
Conversion Codes	
Subrange extraction	76
Conversion Codes	
Arrays.....	73
Floating point	74
IEEE Floating Point	71
Intel Little endian	70
LCN Definition.....	73
Logical Bit Inversion.....	74
Motorola Big endian	70
Packed Register Writes	75
Scaling factor.....	75
Conversion Codes	
Long Integer	77
Conversion Codes	
Examples.....	78

D

DEB	xi
Accumulator Point	92
Analog Input Point	83
Analog Output Point	87
Array Input Point	112
Array Output Point.....	112
Digital Composite Point.....	104
Digital Input Point	102
Flag Point	106
Message Point	109
Numeric Point.....	100
Totalizer Point	96
DET_STS	134
Diagnostic Loopback Test Data.....	62
Diagnostic Register.....	62
DOWNLOAD	133
DPOINT	135
DPT_AD1.....	135
DPT_AD2.....	135
DWN_STS	133

E

EB <i>xi</i>	
ENT_NUM	133
ENT_SIZ	133, 135
ENTRY	133, 135
Error Codes.....	141
Examples	
Conversion Codes	78
Examples	
Load Directive File	47
PITE Register Mapping.....	69
Typical PIT	63
Examples	
Array Input/Output.....	115
Exception Status Coil.....	62

F

FAIL_AT	134
FAIL_TM	54, 55, 57, 134

G

GUS	<i>xi</i>
-----------	-----------

H

HM	<i>xi</i>
----------	-----------

I

Image point	
Accumulator Point	92
Analog Input Point	83
Analog Output	
Earlier Versions	86
Analog Output Point	87
Array Input	112
Array Output.....	112
Design and configuration process	65
Digital Composite Point	104
Digital Input Point	102
Flag Point	106
Message Point.....	109
Numeric Point.....	100
Totalizer Point	96
Installation	
NT install	29, 36
TPS/CLM to APP	26
Intended audience	14

K

Key parameters	134
----------------------	-----

L

LCN.....	<i>xi</i>
----------	-----------

M

Message Point	
Displays	129
Message Point.....	109
Display Interface	128
Modbus	
Function code.....	60
Interface package items	23
Modbus Address Ranges	68
Modbus Addressable components	50
Modbus Data Types.....	68
Modbus Extended Address Ranges	67
Modbus <i>Master</i>	58
Modbus <i>Slave</i>	58
MVIP	v, <i>xi</i>

N

NAME	133, 135
NCF	<i>xi</i>
NDT.....	<i>xi</i>
CLMI/CCP	53
CLMI/CCP BIF	53
CLMI/CCP Status Detail Display	61
Define NDT structure	20
Device BIF	59
Device Status Detail Display	62
Explanation of TDF Lines	133
Hierarchy	50
Port	54, 57
Port BIF	55
Port Status Detail Display	61
User modified device TDF	52
NDT_PO.....	135
NUM_KEYS.....	133, 135
NUM_PARS	135

O

OPT	133
-----------	-----

P

PIT.....	<i>xi</i>
PIT_NAME.....	135

PITE	
Output Parameter Exceptions.....	80
PITE.....	<i>xi</i>
Input Parameter Exceptions	80
Input parameter options	80
Output parameter options	80
PITE	
Template assumptions	81
PITE	
Template Definition File templates in Appendix B ..	135
PITE	
Explanation of TDF lines repeated for each PITEP...	136
PITEP	
Pnn_AD1	136
Pnn_AD2	136
Pnn_DPAR.....	136
Pnn_ICNV.....	136
Pnn_ID.....	136
Pnn_IDST	137
Pnn_IFNC	136
CN	136
CYC.....	136
ECP.....	136
PPS	136
RBI	136
SB136	
SLO	136
SYN.....	136
Pnn_ISTA.....	136
Pnn_ITVO.....	137
Pnn_OCNV	137
Pnn_ODST.....	137
Pnn_OFNC.....	137
CBO.....	137
CYC.....	137
ECP.....	137
PPS	137
RB	137
SLO	137
SYN.....	137
Pnn_OSTA.....	137
Pnn_OTVO	137
Pnn_PAR	136
Pnn_QUAL	136
PITG	<i>xi</i>
PPS	<i>xi</i> , 135
PPS Option.....	81

Prerequisite knowledge.....	14
PT_ID.....	135
PT_OPT	135

R

RB_TIME.....	135
RBE	<i>xi</i> , 133
Reference documents	131
RTU	<i>xi</i>

S

STA.....	134
Status Display.....	62
STS	134
STSDF	<i>xi</i>

T

TBL_TYPE	133, 135
TDC	<i>xi</i>
TDC Emulators Service	
Starting the service	32, 39
Stopping the service	27, 34
TDF.....	<i>xi</i>
PIT, PITG	63
TPN.....	<i>xi</i>
TPS	<i>xi</i>

U

UP_STS.....	133
UPLOAD.....	133
US <i>xi</i>	

W

What this manual does	13
------------------------------------	----

Z

ZC <i>xi</i>	
ZC file.....	44
AM/CLM.....	46
TPS/CLM	45
ZI <i>xi</i>	
ZS <i>xi</i>	
ZT <i>xi</i>	

Honeywell

Honeywell Process Solutions
1860 W. Rose Garden Lane
Phoenix, AZ 85027 US